



TXW80X API manual



Notice

Due to product version upgrades or other reasons, this document will be updated from time to time. Unless otherwise agreed, this document only
As a guide for use without any warranty.

Zhuhai Taixin
Semiconductor Co.,Ltd

3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City

Confidentiality level	A	TXW80X API manual	File No	TX-0000
Release date 2022-05-20			file version	V1.0

Revision history

date	Version	describe	Revised by
2022-05-20	V1.0	initial version	

Confidentiality level	A	TXW80X API manual	File No	TX-0000
Release date	2022-05-20		file version	V1.0

Table

of Contents TXW80X API Manual.....

1 1. GPIO interface illustrate..... 1

1.1. Enum..... 1

1.1.1. gpio_pin_direction.....1 1.1.2.

gpio_pin_mode.....1 1.1.3.

gpio_irq_event.....2 1.1.4.

gpio_iocctl_cmd.....2 1.1.5.

gpio_pull_level.....2 1.1.6.

pin_driver_strength.....3 1.1.7.

gpio_afio_set.....4 1.1.8.

gpio_iomap_out_func.....4 1.1.9.

gpio_iomap_in_func.....5

1.2. Define..... 5

1.2.1. MACRO..... 5

1.3. Structure..... 6

1.3.1. gpio_device..... 6

1.4. Function..... 6

1.4.1. gpio_set_mode().....6 1.4.2.

gpio_set_dir().....7 1.4.3.

gpio_set_val().....8 1.4.4.

gpio_get_val().....9 1.4.5.

gpio_driver_strength().....10 1.4.6.

gpio_set_altn_func().....11 1.4.7.

gpio_iomap_output().....12 1.4.8.

gpio_iomap_input().....13 1.4.9.

gpio_iomap_inout().....14 1.4.10.

gpio_request_pin_irq().....15 1.4.11.

gpio_release_pin_irq().....16 1.4.12.

gpio_iocctl().....17 2. UART 2.1. Enum..... 18

2.1.1. uart_mode..... 18

2.1.2. uart_parity.....18

2.1.3. uart_stop_bit.....19 2.1.4.

uart_data_bit.....19 2.1.5.

uart_irq_flag.....19 2.1.6.

uart_iocctl_cmd.....20

Confidentiality level	A	TXW80X API manual	File No	TX-0000
Release date	2022-05-20		file version	V1.0

2.2. Define.....	21
2.2.1. MACRO.....	21
2.3. Structure.....	21
2.3.1. uart_device.....	21
2.4. Function.....	22
2.4.1. uart_open().....	22
2.4.2. uart_close().....	23
2.4.3. uart_putc().....	24
2.4.4. uart_getc().....	24
2.4.5. uart_puts().....	25
2.4.6. uart_gets().....	26
2.4.7. uart_ioctl().....	27
2.4.8. uart_request_irq().....	28
2.4.9. uart_release_irq().....	29
3. I2S 寄存器.....	31
3.1. Enum.....	31
3.1.1. i2s_sample_bits.....	31
3.1.2. i2s_sample_freq.....	31
3.1.3. i2s_channel.....	31
3.1.4. i2s_data_fmt.....	32
3.1.5. i2s_mode.....	32
3.1.6. i2s_ioctl_cmd.....	32
3.1.7. i2s_irq_flag.....	33
3.2. Define.....	33
3.2.1. MACRO.....	33
3.3. Structure.....	33
3.3.1. i2s_device.....	33
3.4. Function.....	34
3.4.1. i2s_open().....	34
3.4.2. i2s_close().....	35
3.4.3. i2s_write().....	36
3.4.4. i2s_read().....	37
3.4.5. i2s_ioctl().....	38
3.4.6. i2s_request_irq().....	39
3.4.7. i2s_release_irq().....	41
4. PDM 寄存器.....	42
4.1. Enum.....	42
4.1.1. pdm_sample_freq.....	42
4.1.2. pdm_channel.....	42
4.1.3. pdm_irq_flag.....	42

Confidentiality level	A	TXW80X API manual	File No	TX-0000
Release date	2022-05-20		file version	V1.0

4.1.4. pdm_ioctl_cmd.....	43
4.2. Define.....	43
4.2.1. MACRO.....	43
4.3. Structure.....	43
4.3.1. pdm_device.....	43
4.4. Function.....	44
4.4.1. pdm_open().....	44
4.4.2. pdm_read().....	45
4.4.3. pdm_close().....	46
4.4.4. pdm_request_irq().....	47
4.4.5. pdm_release_irq().....	48
4.4.6. pdm_ioctl().....	49
5. ADC 寄存器.....	50
5.1. Enum.....	50
5.1.1. adc_irq_flag.....	50
5.1.2. adc_ioctl_cmd.....	50
5.1.3. adc_voltage_type.....	50
5.2. Define.....	51
5.2.1. MACRO.....	51
5.3. Structure.....	51
5.3.1. adc_device.....	51
5.4. Function.....	52
5.4.1. adc_open().....	52
5.4.2. adc_close().....	53
5.4.3. adc_add_channel().....	54
5.4.4. adc_delete_channel().....	55
5.4.5. adc_get_value().....	56
5.4.6. adc_ioctl().....	57
5.4.7. adc_request_irq().....	58
5.4.8. adc_release_irq().....	59
6. DVP 寄存器.....	61
6.1. Enum.....	61
6.1.1. dvp_ioctl_cmd.....	61
6.2. Define.....	62
6.3. Structure.....	62
6.3.1. dvp_device.....	62
6.4. Function.....	63
6.4.1. dvp_init().....	63
6.4.2. dvp_open().....	63
6.4.3. dvp_close().....	64

Confidentiality level	A	TXW80X API manual	File No	TX-0000
Release date	2022-05-20		file version	V1.0

6.4.4. dvp_set_baudrate().....	65	6.4.5.	
dvp_set_size().....	66	6.4.6. dvp_set_addr1()/	
dvp_set_addr2().....	67	6.4.7.	
dvp_set_rgb2yuv().....	68	6.4.8.	
dvp_set_format().....	69	6.4.9.	
dvp_set_half_size().....	70	6.4.10.	
dvp_set_vsync_polarity()/dvp_set_hsync_polarity().....	71	6.4.11.	
dvp_set_once_sampling().....	72	6.4.12.	
dvp_debounce_enable().....	73	6.4.13.	
dvp_set_ycbcr().....	74	6.4.14.	
dvp_unload_uv().....	75	6.4.15.	
dvp_frame_load_precent().....	76	6.4.16.	
dvp_low_high_threshold().....	77	6.4.17.	
dvp_set_exchange_d5_d6().....	78	6.4.18.	
dvp_jpeg_mode_set_len().....	79	6.4.19.	
dvp_request_irq().....	80	6.4.20.	
dvp_release_irq().....	81	7. JPG	yyy
ÿ.....	82		
7.1. Enum.....	82		
7.1.1. jpeg_ioctl_cmd.....	82		
7.2. Define.....	83		
7.3. Structure.....	83		
7.3.1. jpeg_device.....	83	7.4.	
Function.....	84		
7.4.1. jpeg_init().....	84	7.4.2.	
jpeg_open().....	85		
7.4.3. jpeg_close().....	86		
7.4.4. jpeg_updata_dqt().....	87		
7.4.5. jpeg_set_qt()... ..	88		
7.4.6. jpeg_set_size().....	89	7.4.7.	
jpeg_set_addr().....	90	7.4. 8.	
jpeg_request_irq().....	91	7.4.9.	
jpeg_release_irq().....	93	8. CRC interface	
description.....	94		
8.1. Enum.....	94		
8.1.1. CRC_DEV_FLAGS.....	94	8.1.2.	
CRC_DEV_TYPE.....	94		
8.2. Define.....	95		
8.2.1. MACRO.....	95		
8.3. Structure.....	95		

Confidentiality level	A	TXW80X API manual	File No	TX-0000
Release date	2022-05-20		file version	V1.0

8.3.1. crc_dev..... 95

8.4. Function..... 96

8.4.1. crc_dev_hold().....96 8.4.2.

8.4.2. crc_dev_calc().....96 9. AES interface

description.....98

9.1. Enum..... 98

9.1.1. SYSAES_KEY_LEN.....98 9.1.2.

9.1.2. SYSAES_MODE.....99 9.2.

Define..... 99

9.2.1. MACRO..... 99

9.3. Structure..... 99

9.3.1. sysaes_para.....99 9.3.2.

9.3.2. sysaes_dev.....100 9.4.

Function..... 100

9.4.1. sysaes_encrypt().....100 9.4.2.

9.4.2. sysaes_decrypt().....101 10. SDHOST interface

description..... 102

10.1. Enum..... 102

10.2. Define..... 102

10.2.1. MACRO..... 102

10.3. Structure..... 103

10.3.1. sdh_device.....103

10.4. Function..... 104

10.4.1. sdhost_io_func_init().....104

10.4.2. sdhost cmd.....105

10.4.3. sd_multiple_read().....106 10.4.4.

10.4.4. sd_multiple_write().....107 10.4.5.

10.4.5. sd_set_clk().....109 10.4.6.

10.4.6. sd_set_bus_width().....110 10.4.7. Sd

10.4.7. Sd open..... 110 10.4.8. Sd

10.4.8. Sd close.....111

11. I2C interface description..... 113

11.1. Enum..... 113

11.1.1. i2c_mode.....113 11.1.2.

11.1.2. i2c_addr_mode.....113 11.1.3.

11.1.3. i2c_ioctl_cmd.....113 11.1.4.

11.1.4. i2c_irq_flag.....114

11.2. Define..... 114

11.2.1. MACRO..... 114

11.3. Structure..... 114

Confidentiality level	A	TXW80X API manual	File No	TX-0000
Release date	2022-05-20		file version	V1.0

11.3.1. i2c_device.....	114
11.4. Function.....	115
11.4.1. i2c_open().....	115
11.4.2. i2c_close().....	116
11.4.3. i2c_set_baudrate().....	117
11.4.4. i2c_ioctl().....	118
11.4.5. i2c_write().....	119
11.4.6. i2c_read().....	120
11.4.7. i2c_request_irq().....	122
11.4.8. i2c_release_irq().....	123
12. SPI 寄存器.....	125
12.1. Enum.....	125
12.1.1. spi_work_mode.....	125
12.1.2. spi_wire_mode.....	125
12.1.3. spi_clk_mode.....	126
12.1.4. spi_ioctl_cmd.....	126
12.1.5. spi_irq_flag.....	126
12.2. Define.....	127
12.2.1. MACRO.....	127
12.3. Structure.....	127
12.3.1. spi_device.....	127
12.4. Function.....	128
12.4.1. spi_open().....	128
12.4.2. spi_close().....	129
12.4.3. spi_ioctl().....	130
12.4.4. spi_read().....	131
12.4.5. spi_write().....	132
12.4.6. spi_set_cs().....	133
12.4.7. spi_request_irq().....	135
12.4.8. spi_release_irq().....	136
13. TIMER 寄存器.....	138
13.1. Enum.....	138
13.1.1. timer_type.....	138
13.1.2. timer_irq_flag.....	138
13.1.3. timer_ioctl_cmd.....	138
13.2. Define.....	139
13.2.1. MACRO.....	139
13.3. Structure.....	139
13.3.1. timer_device.....	139
13.4. Function.....	140

Confidentiality level	A	TXW80X API manual	File No	TX-0000
Release date	2022-05-20		file version	V1.0

13.4.1. timer_device_open().....140 13.4.2.

timer_device_close().....141 13.4.3.

timer_device_start().....141 13.4.4.

timer_device_stop().....143 13.4.5.

timer_device_ioctl().....144 13.4.6.

timer_device_request_irq().....145 13.4.7.

timer_device_release_irq().....146 14. PWM 14.1. PWM 14.1.1. Enum..... 147

14.1.1. pwm_channel.....147 14.1.2.

pwm_irq_flag.....147 14.1.3.

pwm_ioctl_cmd.....147

14.2. Define..... 148

14.2.1. MACRO..... 148

14.3. Structure..... 148

14.3.1. pwm_device.....148

14.4. Function..... 149

14.4.1. pwm_init().....149

14.4.2. pwm_deinit().....150

14.4.3. pwm_start()... ..151

14.4.4. pwm_stop().....152 14.4.5.

pwm_ioctl()..... .153

14.4.6. pwm_request_irq().....154

14.4.7. pwm_release_irq().....156 15.

CAPTURE interface description..... 158

15.1. Enum..... 158

15.1.1. capture_channel.....158 15.1.2.

capture_mode.....158 15.1.3.

capture_irq_flag.....158 15.1.4.

capture_ioctl_cmd.....159

15.2. Define..... 159

15.2.1. MACRO..... 159

15.3. Structure..... 159

15.3.1. capture_device.....159

15.4. Function..... 160

15.4.1. capture_init().....160 15.4.2.

capture_deinit().....161 15.4.3.

capture_start().....162 15.4.4.

capture_stop().....163 15.4.5.

capture_ioctl().....164

Confidentiality level	A	TXW80X API manual	File No	TX-0000
Release date 2022-05-20			file version	V1.0

15.4.6. capture_request_irq().....165

15.4.7. capture_release_irq().....166

1. GPIO interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of GPIO. They are located in the "" file directory

1.1. Enum

1.1.1. gpio_pin_direction

enumerator	illustrate
GPIO_DIR_INPUT	Set GPIO PIN to normal input mode
GPIO_DIR_OUTPUT	Set GPIO PIN to normal output mode

1.1.2. gpio_pin_mode

enumerator	illustrate
GPIO_PULL_NONE	Set GPIO PIN to no pull-down mode
GPIO_PULL_UP	Set GPIO PIN to pull-up mode
GPIO_PULL_DOWN	Set GPIO PIN to pull-down mode
GPIO_OPENDRAIN_PULL_NONE	Set GPIO PIN to open-drain no pull-up mode
GPIO_OPENDRAIN_PULL_UP	Set GPIO PIN to open-drain pull-up mode
GPIO_OPENDRAIN_DROP_NONE	Set GPIO PIN to open-drain no pull-down mode
GPIO_OPENDRAIN_DROP_DOWN	Set GPIO PIN to open-drain pull-down mode

1.1.3. gpio_irq_event

enumerator	illustrate
GPIO_IRQ_EVENT_NONE	Setup without interruption
GPIO_IRQ_EVENT_RISE	Set rising edge interrupt
GPIO_IRQ_EVENT_FALL	Set falling edge interrupt
GPIO_IRQ_EVENT_ALL	Set level interrupt

1.1.4. gpio_iocctl_cmd

enumerator	illustrate
GPIO_INPUT_LAG	Set GPIO PIN input hysteresis @Note: Input hysteresis: The voltage values that define 0 and 1 change from a fixed value to a range value (a range above and below the fixed value), circumference). For example, the fixed voltage value of 0 and 1 is 2V, and after turning on the hysteresis, it becomes 1.8V~2.2V. Therefore, the voltage greater than 2.2V is 1, and the voltage less than 1.8V is 0.
GPIO_DIR_ATOMIC	Atomic operation to set GPIO PIN direction
GPIO_VALUE_ATOMIC	Atomic operation to set GPIO PIN output value
GPIO_LOCK	The GPIO is locked and the configuration cannot be changed unless the GPIO module is reset.
GPIO_DEBUNCE	Turn on the filtering function of GPIO PIN input
GPIO_OUTPUT_TOGGLE	Negate the GPIO PIN output value
GPIO_GENERAL_ANALOG	Set GPIO PIN to normal simulation mode

1.1.5. gpio_pull_level

enumerator	illustrate
GPIO_PULL_LEVEL_NONE	Set the GPIO PIN pull-up/down resistance value to 0

GPIO_PULL_LEVEL_4_7K	Set the GPIO PIN pull-up/down resistance value to 4.7K
GPIO_PULL_LEVEL_10K	Set the GPIO PIN pull-up/down resistance value to 10K
GPIO_PULL_LEVEL_100K	Set the GPIO PIN pull-up/down resistance value to 100K

1.1.6. pin_driver_strength

enumerator	illustrate
GPIO_DS_4MA	Set the driving capability of GPIO PIN to 4mA
GPIO_DS_8MA	Set the driving capability of GPIO PIN to 8mA
GPIO_DS_12MA	Set the driving capability of GPIO PIN to 12mA
GPIO_DS_16MA	Set the driving capability of GPIO PIN to 16mA
GPIO_DS_20MA	Set the driving capability of GPIO PIN to 20mA
GPIO_DS_24MA	Set the driving capability of GPIO PIN to 24mA
GPIO_DS_28MA	Set the driving capability of GPIO PIN to 28mA
GPIO_DS_32MA	Set the driving capability of GPIO PIN to 32mA
GPIO_DS_36MA	Set the driving capability of GPIO PIN to 36mA
GPIO_DS_40MA	Set the driving capability of GPIO PIN to 40mA
GPIO_DS_44MA	Set the driving capability of GPIO PIN to 44mA
GPIO_DS_48MA	Set the driving capability of GPIO PIN to 48mA
GPIO_DS_52MA	Set the driving capability of GPIO PIN to 52mA
GPIO_DS_56MA	Set the driving capability of GPIO PIN to 56mA
GPIO_DS_60MA	Set the driving capability of GPIO PIN to 60mA
GPIO_DS_64MA	Set the driving capability of GPIO PIN to 64mA

1.1.7. gpio_afio_set

enumerator	illustrate
GPIO_AF_0	Set the AFIO value of GPIO PIN to 0
GPIO_AF_1	Set the AFIO value of GPIO PIN to 1
GPIO_AF_2	Set the AFIO value of GPIO PIN to 2
GPIO_AF_3	Set the AFIO value of GPIO PIN to 3

1.1.8. gpio_iomap_out_func

enumerator	illustrate

1.1.9. gpio_iomap_in_func

enumerator	illustrate

1.2. Define

1.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below, if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

1.3. Structure

1.3.1. gpio_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
mode	int32 function pointer	
you	int32 function pointer	
set	int32 function pointer	
get	int32 function pointer	
request_pin_irq	int32 function pointer	
release_pin_irq	int32 function pointer	
ioctl	int32 function pointer	

1.4. Function

1.4.1. gpio_set_mode()

Set the working mode of the GPIO PIN and the pull-up or pull-down resistance value in this mode.

Function prototype

```
int32 gpio_set_mode( uint32 pin,
                    enum gpio_pin_mode mode,
                    enum gpio_pull_level level
                    )
```

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
mode	enum gpio_pin_mode	sets the mode of GPIO PIN, see enumeration gpio_pin_mode
level	enum gpio_pull_level	sets the pull-up or pull-down resistance of the GPIO PIN, see enumeration gpio_pull_level

Return value

return value	type	illustrate
RET_OK	int32	GPIO PIN mode and resistance value configured successfully
RET_ERR	int32	GPIO PIN mode and resistance configuration failed
- SINGLE CHOICE	int32	GPIO PIN mode and resistance configuration failed due to incorrect function parameters.

Code examples

```
/* Configure PA1 in pull-up mode, with a pull-up resistance value of 10K */

gpio_set_mode(PA_1, GPIO_PULL_UP, GPIO_PULL_LEVEL_10K);
```

1.4.2. gpio_set_dir()

Set the direction of the GPIO PIN.

Function prototype

int32 gpio_set_dir(uint32 pin, enum gpio_pin_direction direction)

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
direction	enum gpio_pin_direction	sets the direction of the GPIO PIN, see enum gpio_pin_direction

Return value

return value	type	illustrate
RET_OK	int32	GPIO PIN direction set successfully
RET_ERR	int32	Direction setting of GPIO PIN failed
- SINGLE CHOICE	int32	The setting direction of the GPIO PIN failed because the function parameters were incorrect.

Code examples

```
/*Configure PA1 as normal output mode*/

gpio_set_dir(PA_1, GPIO_DIR_OUTPUT);
```

1.4.3. gpio_set_val()

Set the value of GPIO PIN. Before calling this function, the pin must be set to normal output mode, see function gpio_set_dir()

Function prototype

int32 gpio_set_val(uint32 pin, int32 value)

Function parameters

parameter	type	illustrate
-----------	------	------------

pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
value	int32	Set the value of GPIO PIN

Return value

return value	type	illustrate
RET_OK	int32	GPIO PIN value set successfully
RET_ERR	int32	GPIO PIN value setting failed
- SINGLE CHOICE	int32	The GPIO PIN value setting failed because the function parameters were incorrectly passed.

Code examples

```
/*Configure PA1 as normal output mode*/

gpio_set_dir(PA_1, GPIO_DIR_OUTPUT);

/* Configure PA1 output 1 (high level) */

gpio_set_val(PA_1, 1);
```

1.4.4. gpio_get_val()

Get the current value of GPIO PIN. Before calling this function, the pin must be set to normal input mode, see function `gpio_set_dir()`

Function prototype

```
int32 gpio_get_val(uint32 pin, int32 value)
```

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
value	int32	Set the value of GPIO PIN

Return value

return value	type	illustrate
RET_OK	int32	GPIO PIN value set successfully
RET_ERR	int32	GPIO PIN value setting failed
- SINGLE CHOICE	int32	The GPIO PIN value setting failed because the function parameters were incorrectly passed.

Code examples

```
/*Configure PA1 as normal output mode*/

gpio_set_dir(PA_1, GPIO_DIR_OUTPUT);

/* Configure PA1 output 1 (high level) */

gpio_set_val(PA_1, 1);
```

1.4.5. gpio_driver_strength()

Set the driving capability of GPIO PIN.

Function prototype

```
int32 gpio_driver_strength(uint32 pin, enum pin_driver_strength strength)
```

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
strength	enum pin_driver_strength	sets the driving capability of GPIO PIN, see enumeration pin_driver_strength

Return value

return value	type	illustrate
RET_OK	int32	GPIO PIN driver capability set successfully
RET_ERR	int32	GPIO PIN driver capability setting failed

- SINGLE CHOICE	int32	Because the function parameters are incorrect, the GPIO PIN driver capability setting fails.
-----------------	-------	--

Code examples

```
/* Configure the driving capability of PA1 to 32mA */
gpio_driver_strength(PA_1, GPIO_DS_32MA);
```

1.4.6. gpio_set_afio_func()

Set the AFIO of the GPIO PIN and configure it according to the AFIO table of the GPIO.

Function prototype

```
int32 gpio_set_afio_func(uint32 pin, enum gpio_afio_set afio)
```

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
live	and gpio_afio_set	Set the AFIO of GPIO PIN, see enumeration gpio_afio_set

Return value

return value	type	illustrate
RET_OK	int32	AFIO setting of GPIO PIN successful
RET_ERR	int32	AFIO setting of GPIO PIN failed
- SINGLE CHOICE	int32	The AFIO setting of the GPIO PIN failed because the function parameters were incorrectly passed.

Code examples

```
/* Configure the AFIO of PA1 to 0 */

gpio_set_altnfunc(PA_1, GPIO_AF_0);
```

1.4.7. gpio_iomap_output()

Set the function of IOMAP OUTPUT of GPIO PIN and configure it according to the required function.

Function prototype

```
int32 gpio_iomap_output(uint32 pin, enum gpio_iomap_out_func func_sel)
```

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
func_sel	enum gpio_iomap_out_func	Set the iomap output function of the GPIO PIN, see enumeration gpio_iomap_out_func

Return value

return value	type	illustrate
RET_OK	int32	The iomap output function of GPIO PIN is set successfully
RET_ERR	int32	GPIO PIN iomap output function setting failed
- SINGLE CHOICE	int32	The iomap output function setting of the GPIO PIN failed due to an error in the function parameters.

Code examples

```

/* Configure the function of iomap_output of PA1 as TX of UART0 */

/* That is, PA1 serves as the TX function of UART0*/

gpio_iomap_output(PA_1, GPIO_IOMAP_OUT_UART0_OUT);

```

1.4.8. gpio_iomap_input()

Set the function of IOMAP INPUT of GPIO PIN and configure it according to the required function.

Function prototype

```
int32 gpio_iomap_input(uint32 pin, enum gpio_iomap_in_func func_sel)
```

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
func_sel	enum gpio_iomap_in_func	Set the iomap input function of GPIO PIN, see enumeration <code>gpio_iomap_in_func</code>

Return value

return value	type	illustrate
RET_OK	int32	The iomap input function of GPIO PIN is set successfully
RET_ERR	int32	GPIO PIN iomap input function setting failed
- SINGLE CHOICE	int32	The iomap input function setting of GPIO PIN failed due to incorrect function parameters.

Code examples

```
/* Configure the iomap_output function of PA1 to be the RX of UART0 */

/* That is, PA1 serves as the RX function of UART0*/

gpio_iomap_output(PA_1, GPIO_IOMAP_OUT_UART0_IN);
```

1.4.9. gpio_iomap_inout()

Set the function of IOMAP INOUT of GPIO PIN and configure it according to the required function. For example, the SPI module All I/Os are required to support input and output, so use this function to configure the GPIO PIN to INOUT function.

Function prototype

```
int32 gpio_iomap_inout( uint32 pin,

                      enum gpio_iomap_in_func in_func_sel,

                      enum gpio_iomap_out_func out_func_sel

                      )
```

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
in_func_sel	enum gpio_iomap_in_func	Set the iomap input function of GPIO PIN, see enumeration gpio_iomap_in_func
out_func_sel	enum gpio_iomap_out_func	Set the iomap output function of the GPIO PIN, see enumeration gpio_iomap_out_func

Return value

return value	type	illustrate
RET_OK	int32	The iomap inout function of GPIO PIN is set successfully
RET_ERR	int32	GPIO PIN iomap inout function setting failed
- SINGLE CHOICE	int32	The iomap inout function setting of the GPIO PIN failed because the function parameters were incorrectly passed.

Code examples

```
/* Configure the function of iomap_inout of PA1 as CLK of SPI0 */

/* That is, PA1 serves as the CLK function of SPI0*/

gpio_iomap_inout( PA_1,

                  GPIO_IOMAP_IN_SPI0_SCK_IN,

                  GPIO_IOMAP_OUT_SPI0_SCK_OUT);
```

1.4.10. gpio_request_pin_irq()

Apply for GPIO PIN interrupt.

Function prototype

```
int32 gpio_request_pin_irq( uint32 pin,

                           gpio_irq_hdl handler,

                           uint32 data,

                           enum gpio_irq_event evt

                           )
```

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
handler	gpio_irq_hdl	Interrupt handle, executed after the interrupt is generated
data	uint32	Interrupt handle parameters
Possibly	enum gpio_irq_event	The requested interrupt type, refer to enumeration gpio_irq_event

Return value

return value	type	illustrate
RET_OK	int32	GPIO PIN application interrupted successfully
RET_ERR	int32	GPIO PIN application interrupt failed

- SINGLE CHOICE	int32	The application for GPIO PIN interrupt failed because the function parameters were incorrect.
-----------------	-------	---

Code examples

```
void pin_irq_test_handle(int32 id, enum gpio_irq_event evt) {
    __NOP();
}

/*PA1 applies for rising edge interrupt*/

gpio_request_pin_irq( PA_1,
                    (gpio_irq_hdl)pin_irq_test_handle,
                    0,
                    GPIO_IRQ_EVENT_RISE);
```

1.4.11. gpio_release_pin_irq()

Release GPIO PIN interrupt.

Function prototype

```
int32 gpio_release_pin_irq(uint32 pin, enum gpio_irq_event evt)
```

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
Possibly	enum gpio_irq_event	The type of interrupt released, refer to the enumeration gpio_irq_event

Return value

return value	type	illustrate
RET_OK	int32	GPIO PIN release interrupt successful
RET_ERR	int32	Release interrupt of GPIO PIN failed
- SINGLE CHOICE	int32	The release interrupt of the GPIO PIN failed because the function parameters were incorrect.

Code examples

```
/*PA1 releases rising edge interrupt*/

gpio_release_pin_irq(PA_1, GPIO_IRQ_EVENT_RISE);
```

1.4.12. gpio_iocfl()

According to the command in enum gpio_iocfl_cmd, call this function to configure the GPIO.

Function prototype

```
int32 gpio_iocfl(uint32 pin, int32 cmd, int32 param1, int32 param2)
```

Function parameters

parameter	type	illustrate
pin	uint32	GPIO PIN, currently supports GPIOA, GPIOB, and GPIOC pins
cmd	int32	For configuration commands of the GPIO module, see enumeration gpio_iocfl_cmd
param1	uint32	Configuration parameter 1, determined according to the configuration command
param2	uint32	Configuration parameter 2, determined according to the configuration command

Return value

return value	type	illustrate
RET_OK	int32	GPIO module configured successfully
RET_ERR	int32	GPIO module configuration failed
- SINGLE CHOICE	int32	GPIO configuration failed due to incorrect function parameters.

Code examples

```
/*Configure PA1 as normal output mode*/

gpio_set_dir(PA_1, GPIO_DIR_OUTPUT);

/* Configure the PA1 output value to flip once */

gpio_ioctl(PA_1, GPIO_OUTPUT_TOGGLE, 1, 0);
```

2. UART interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of UART. They are located in the "" file directory

2.1. Enum

2.1.1. uart_mode

enum uart_mode describes the working mode of UART.

enumerator	illustrate
UART_MODE_DUPLEX	UART works in full-duplex mode, that is, it can receive and send data at the same time
UART_MODE_SIMPLEX_TX	The UART works in single-send mode, that is, the UART only performs the action of sending data. Depending on the data received @Note: In this mode, the TX pin is used for transmission and data changes on the RX pin are ignored.
UART_MODE_SIMPLEX_RX	The UART works in single receiving mode, that is, the UART only performs the action of data reception. Depending on the data sent @Note: In this mode, the RX pin is used for reception and data changes on the TX pin are ignored.

2.1.2. uart_parity

enum uart_parity describes the UART parity mode.

enumerator	illustrate
------------	------------

UART_PARITY_NONE	UART works in no-verify mode
UART_PARITY_ODD	UART works in odd parity mode. The bit of each byte is 1 during the entire transmission process. If the number of is an odd number, the check digit is 1, otherwise it is 0
UART_PARITY_EVEN	The UART works in even parity mode. The bit of each byte is 1 during the entire transmission process. If the number of is an even number, the check digit is 1, otherwise it is 0

2.1.3. uart_stop_bit

enum uart_stop_bit describes the number of UART stop bit bits.

enumerator	illustrate
UART_STOP_BIT_1	The stop bit of UART is 1bit
UART_STOP_BIT_2	The stop bit of UART is 2bit

2.1.4. uart_data_bit

enum uart_data_bit describes the number of UART stop bit bits.

enumerator	illustrate
UART_DATA_BIT_8	The data bits of UART are 8 bits
The data bits of UART are 9 bits	The data bits of UART are 9 bits @Note: In this mode, the UART verification mode must be no verification mode

2.1.5. uart_irq_flag

enum uart_irq_flag describes the interrupt types that UART can apply for, through uart_request_irq() and uart_release_irq() function is used.

enumerator	illustrate
UART_IRQ_FLAG_TX_BYTE	After UART sends one frame of data, an interrupt is generated. @Note: If the UART configuration data bit is 8bit, one frame of data is 8bit; if the data bit is 9bit, one frame of data is 9bit

UART_IRQ_FLAG_TIME_OUT	After the UART exceeds the set timeout time and has not received data, an interrupt is generated. @Note: This interrupt is only used for UART reception
UART_IRQ_FLAG_DMA_TX_DONE	UART uses DMA to send, and an interrupt is generated. @Note: Before configuring this interrupt, you need to use uart_iocfl to enable DMA
UART_IRQ_FLAG_DMA_RX_DONE	When UART uses DMA to receive, an interrupt is generated. @Note: Before configuring this interrupt, you need to use uart_iocfl to enable DMA
UART_IRQ_FLAG_FRAME_ERR	If the UART receives a frame of data error, an interrupt will be generated. @Note: 1. If the UART configuration data bit is 8bit, one frame of data is 8bit; if the data bit is 9bit, one frame of data is 9bit 2. A frame of data error usually means that the number of stop bits in a frame of data does not match the set number of bits.
UART_IRQ_FLAG_RX_BYTE	After UART receives a frame of data, an interrupt is generated. @Note: If the UART configuration data bit is 8bit, one frame of data is 8bit; if the data bit is 9bit, one frame of data is 9bit

2.1.6. uart_iocfl_cmd

enum uart_iocfl_cmd describes the UART configuration command, which is configured by calling uart_iocfl().

enumerator	illustrate
UART_IOCTL_CMD_SET_BAUDRATE	Set UART baud rate
UART_IOCTL_CMD_SET_DATA_BIT	Set the number of data bits of UART @Note: Reference enum uart_data_bit
UART_IOCTL_CMD_SET_PARITY	Set UART verification mode @Note: Reference enum uart_parity
UART_IOCTL_CMD_SET_STOP_BIT	Set the number of stop bits of UART @Note: Reference enum uart_stop_bit
UART_IOCTL_CMD_SET_TIME_OUT	Set UART reception timeout
UART_IOCTL_CMD_SET_DMA	Set whether UART uses DMA to send @Note: Set to use DMA transfer, only effective for uart_puts() and uart_gets() functions

UART_IOCTL_CMD_SET_WORK_MODE	Set UART working mode @Note: Reference enum uart_mode
------------------------------	---

2.2. Define

2.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

2.3. Structure

2.3.1. uart_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
open	int32 function pointer	
close	int32 function pointer	
putc	int32 function pointer	
getc	int32 function pointer	
puts	int32 function pointer	
gets	int32 function pointer	

ioctl	int32 function pointer	
request_irq	int32 function pointer	
release_irq	int32 function pointer	

2.4. Function

2.4.1. uart_open()

Initialize UART. This function must be called before using the UART module. After UART initialization is successful, the default configuration Set to: The working mode is full-duplex mode, the parity mode is no parity mode, the data bit is 8bit, and the stop bit is 1bit.

If you need to change the configuration, you can use the `uart_ioctl` function to make changes after the initialization is completed.

Function prototype

```
int32 uart_open(struct uart_device *uart, uint32 baudrate)
```

Function parameters

parameter	type	illustrate
uart	struct uart_device	UART handle. Usually use <code>dev_get()</code> to get the handle
baudrate	uint32	UART baud rate value

Return value

return value	type	illustrate
RET_OK	int32	UART module initialized successfully
RET_ERR	int32	UART module initialization failed

Code examples

```
struct uart_device          *uart_test;

uart_test = (struct uart_device*)dev_get(HG_UART0_DEVID);

/* Get the UART handle through dev_get(), configure the baud rate to 115200, and open the serial port */

uart_open((struct uart_device *)uart_test, 115200);
```

2.4.2. uart_close()

Turn off UART. After calling this function, the UART will not be able to receive and receive data normally, and all configurations (including interrupt-related configurations) settings) will be invalid.

Function prototype

```
int32 uart_close(struct uart_device *uart)
```

Function parameters

parameter	type	illustrate
uart	struct uart_device	UART handle. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	UART module closed successfully
RET_ERR	int32	UART module shutdown failed

Code examples

```
struct uart_device          *uart_test;

uart_test = (struct uart_device*)dev_get(HG_UART0_DEVID);

/* Get the UART handle through dev_get() and close the serial port */

uart_close((struct uart_device *)uart_test);
```

2.4.3. uart_putc()

UART sends a frame of data. If the UART configuration data bit is 8bit, one frame of data is 8bit; if the data bit is 9bit, One frame of data is 9 bits.

Function prototype

```
int32 uart_putc(struct uart_device *uart, int8 value)
```

Function parameters

parameter	type	illustrate
uart	struct uart_device	UART handle. Usually use dev_get() to get the handle
value	you8	Data to be sent by UART, unit is 1 frame

Return value

return value	type	illustrate
RET_OK	int32	UART module initialized successfully
RET_ERR	int32	UART module initialization failed

Code examples

```
struct uart_device      *uart_test;

uint8 tx_data = 0x55;

uart_test = (struct uart_device*)dev_get(HG_UART0_DEVID);

/* Get the UART handle through dev_get(), configure the baud rate to 115200, and open the serial port */
uart_open((struct uart_device *)uart_test, 115200);

/* Send one frame of data (8bit) */

uart_putc((struct uart_device *)uart_test, tx_data);
```

2.4.4. uart_getc()

The UART receives a frame of data. If the UART configuration data bit is 8bit, one frame of data is 8bit; if the data bit is 9bit,

One frame of data is 9 bits.

Function prototype

```
uint8 uart_getc(struct uart_device *uart)
```

Function parameters

parameter	type	illustrate
uart	struct uart_device	UART handle. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
value	uint8	The UART module successfully receives data and returns the data value.

Code examples

```
struct uart_device      *uart_test;

uint8 rx_data;

uart_test = (struct uart_device*)dev_get(HG_UART0_DEVID);

/* Get the UART handle through dev_get(), configure the baud rate to 115200, and open the serial port */
uart_open((struct uart_device *)uart_test, 115200);

/* Receive one frame of data (8bit) */

rx_data = uart_getc((struct uart_device *)uart_test);
```

2.4.5. uart_puts()

UART sends data based on the address of DATA BUFFER and the number of data. The default way for UART to send data is CPU sends. If you want to use DMA to send, you need to use the uart_ioctl() function for configuration.

Function prototype

```
int32 uart_puts(struct uart_device *uart, uint8* buf, uint32 len)
```

Function parameters

parameter	type	illustrate
uart	struct uart_device	UART handle. Usually use dev_get() to get the handle
buf	uint8	Starting address of DATA BUFFER

only	uint32	The number of data to be sent, the default unit is 8bit
------	--------	---

Return value

return value	type	illustrate
RET_OK	int32	UART module sends data successfully
RET_ERR	int32	UART module failed to send data

Code examples

```

struct uart_device      *uart_test;

uint8 tx_data[3] = {1, 2, 3};

uart_test = (struct uart_device*)dev_get(HG_UART0_DEVID);

/* Get the UART handle through dev_get(), configure the baud rate to 115200, and open the serial port */
uart_open((struct uart_device *)uart_test, 115200);

/* Send data (8bit), send 3byte */
uart_puts((struct uart_device *)uart_test, tx_data, 3);

```

2.4.6. uart_gets()

UART receives data based on the address of DATA BUFFER and the number of data. The default way for UART to receive data is CPU reception, if you want to use DMA reception, you need to use the `uart_ioctl()` function for configuration.

Function prototype

```
int32 uart_gets(struct uart_device *uart, uint8* buf, uint32 len)
```

Function parameters

parameter	type	illustrate
uart	struct uart_device	UART handle. Usually use <code>dev_get()</code> to get the handle
buf	uint8	Starting address of DATA BUFFER
only	uint32	The number of data to be received, the default unit is 8bit

Return value

return value	type	illustrate
RET_OK	int32	UART module sends data successfully
RET_ERR	int32	UART module failed to send data

Code examples

```

struct uart_device      *uart_test;
uint8 rx_data[3] = {0};
uart_test = (struct uart_device*)dev_get(HG_UART0_DEVID);

/* Get the UART handle through dev_get(), configure the baud rate to 115200, and open the serial port */
uart_open((struct uart_device *)uart_test, 115200);

/* Send data (8bit), receive 3byte */
uart_gets((struct uart_device *)uart_test, rx_data, 3);

```

2.4.7. uart_ioctl()

According to the command in enum uart_ioctl_cmd, call this function to configure the UART module.

Function prototype

```

int32 uart_ioctl( struct uart_device *uart,
                  enum uart_ioctl_cmd ioctl_cmd,
                  uint32 param1,
                  uint32 param2
                  )

```

Function parameters

parameter	type	illustrate
uart	struct uart_device	UART handle. Usually use dev_get() to get the handle
ioctl_cmd	enum uart_ioctl_cmd	For configuration commands of the UART module, see enumeration uart_ioctl_cmd
param1	uint32	Configuration parameter 1, determined according to the configuration command

param2	uint32	Configuration parameter 2, determined according to the configuration command
--------	--------	--

Return value

return value	type	illustrate
RET_OK	int32	UART module configured successfully
RET_ERR	int32	UART module configuration failed

Code examples

```
struct uart_device *uart_test;
uart_test = (struct uart_device*)dev_get(HG_UART0_DEVID);

/* Get the UART handle through dev_get(), configure the baud rate to 115200, and open the serial port */
uart_open((struct uart_device *)uart_test, 115200);

/* Reset baud rate: 9600 */
uart_ioctl((struct uart_device *)uart_test, UART_IOCTL_CMD_SET_BAUDRATE,
9600, 0);
```

2.4.8. uart_request_irq()

According to the interrupt type of enum uart_irq_flag, call this function to apply for the interrupt of the UART module.

Function prototype

```
int32 uart_request_irq( struct uart_device *uart,
                        uart_irq_hdl irq_hdl,
                        uint32 irq_flag,
                        uint32 irq_data
                        )
```

Function parameters

parameter	type	illustrate
uart	struct uart_device	UART handle. Usually use dev_get() to get the handle
irq_hdl	uart_irq_hdl	Interrupt handle, executed after the interrupt is generated
irq_flag	uint32	The requested interrupt type, refer to the enumeration uart_irq_flag

irq_data	uint32	Interrupt handle parameters
----------	--------	-----------------------------

Return value

return value	type	illustrate
RET_OK	int32	UART module applied for interrupt successfully
RET_ERR	int32	UART module failed to apply for interrupt

Code examples

```
void uart_interrupt_func(int32 data) {
    __NOP();
}

struct uart_device *uart_test;
uart_test = (struct uart_device*)dev_get(HG_UART0_DEVID);

/* Get the UART handle through dev_get(), configure the baud rate to 115200, and open the serial port */
uart_open((struct uart_device *)uart_test, 115200);

/* Apply for receive timeout interrupt, the interrupt handle is uart_interrupt_func, the parameter is 0 */
uart_request_irq((struct uart_device *)uart_test, uart_interrupt_func,
UART_IRQ_FLAG_TIME_OUT, 0);
```

2.4.9. uart_release_irq()

According to the interrupt type of enum uart_irq_flag, call this function to release the interrupt of the UART module.

Function prototype

```
int32 uart_release_irq(struct uart_device *uart, uint32 irq_flag)
```

Function parameters

parameter	type	illustrate
uart	struct uart_device	UART handle. Usually use dev_get() to get the handle
irq_flag	uint32	Released interrupt type, refer to enumeration uart_irq_flag

Return value

return value	type	illustrate
RET_OK	int32	UART module is closed and released successfully
RET_ERR	int32	UART module shutdown release failed

Code examples

```
struct uart_device      *uart_test;

uart_test = (struct uart_device*)dev_get(HG_UART0_DEVID);

/* Get the UART handle through dev_get(), configure the baud rate to 115200, and open the serial port */
uart_open((struct uart_device *)uart_test, 115200);

/* Turn off receive timeout interrupt */

uart_release_irq((struct uart_device *)uart_test,\
UART_IRQ_FLAG_TIME_OUT);
```

3. I2S interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of I2S. They are located in the "" file directory

3.1. Enum

3.1.1. i2s_sample_bits

i2s_sample_bits describes the number of bits of audio data.

enumerator	illustrate
I2S_SAMPLE_BITS_8BITS	I2S outputs 8-bit audio data
I2S_SAMPLE_BITS_16BITS	I2S outputs 16-bit audio data
I2S_SAMPLE_BITS_24BITS	I2S outputs 24-bit audio data

3.1.2. i2s_sample_freq

i2s_sample_freq describes the sampling frequency, that is, the frequency of WSCLK.

enumerator	illustrate
I2S_SAMPLE_FREQ_44_1K	I2S sampling frequency is 44.1KHz
I2S_SAMPLE_FREQ_48K	I2S sampling frequency is 48KHz

3.1.3. i2s_channel

i2s_channel describes the I2S channel mode.

enumerator	illustrate
I2S_CHANNEL_MONO	I2S works in mono mode
I2S_CHANNEL_STEREO	I2S works in two-channel mode

3.1.4. i2s_data_fmt

i2s_data_fmt describes the data format of I2S.

enumerator	illustrate
I2S_DATA_FMT_I2S	The data format of I2S is the standard I2S format.
I2S_DATA_FMT_MSB	The data format of I2S is left-aligned format
I2S_DATA_FMT_LSB	The data format of I2S is right-aligned format
I2S_DATA_FMT_PCM	The data format of I2S is PCM format

3.1.5. i2s_mode

i2s_mode describes the working mode of I2S.

enumerator	illustrate
I2S_MODE_MASTER	I2S works in host mode
I2S_MODE_SLAVE	I2S works in slave mode

3.1.6. i2s_ioctl_cmd

i2s_ioctl_cmd describes the I2S configuration command, which is configured by calling i2s_ioctl().

enumerator	illustrate
I2S_IOCTL_CMD_SET_WSCLK_POL	Set the WSCLK polarity of I2S
I2S_IOCTL_CMD_SET_SAMPLE_BITS	Set the number of bits of I2S audio data @Note: Reference enum i2s_sample_bits
I2S_IOCTL_CMD_SET_CHANNEL	Set the channel mode of I2S @Note: Reference enum i2s_channel
I2S_IOCTL_CMD_SET_DATA_FMT	Set the data format of I2S @Note: Reference enum i2s_data_fmt
I2S_IOCTL_CMD_SET_DEBOUNCE	Set the filter function of I2S @Note:

	This filtering function only acts on the I2S slave; when filtering is turned on, the module will filter SCLK, WSCLK, DATA perform filtering
--	--

3.1.7. i2s_irq_flag

i2s_irq_flag describes the interrupt types that I2S can apply for, through i2s_request_irq() and i2s_release_irq() function is used.

enumerator	illustrate
I2S_IRQ_FLAG_HALF	I2S half-completed transfer interrupt
I2S_IRQ_FLAG_FULL	I2S completes all transfer interrupts

3.2. Define

3.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.
The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

3.3. Structure

3.3.1. i2s_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
open	int32 function pointer	
close	int32 function pointer	

ioctl	int32 function pointer	
read	int32 function pointer	
write	int32 function pointer	
request_irq	int32 function pointer	
release_irq	int32 function pointer	

3.4. Function

3.4.1. i2s_open()

Initialize I2S. This function must be called before using the I2S module. After I2S initialization is successful, the default configuration is: the data format is I2S format; the filtering function is turned off; the left channel of the I2S_WSCLK clock is low level, and the right channel is high level; the channel mode is stereo. If you need to change the configuration, you can use i2s_ioctl after the initialization is completed. function to make changes.

Function prototype

```
int32 i2s_open( struct i2s_device *i2s,
               enum i2s_mode mode,
               enum i2s_sample_freq frequency,
               enum i2s_sample_bits bits
               )
```

Function parameters

parameter	type	illustrate
i2s	struct i2s_device	I2S handle. Usually use dev_get() to get the handle
mode	enum i2s_mode	I2S working mode, refer to enumeration i2s_mode
frequency	enum i2s_sample_freq	I2S sampling frequency, refer to enumeration i2s_sample_freq
bits	enum i2s_sample_bits	The number of audio data bits in I2S, refer to the enumeration i2s_sample_bits

Return value

return value	type	illustrate
RET_OK	int32	I2S module initialization successful
RET_ERR	int32	I2S module initialization failed
- SINGLE CHOICE	int32	The I2S module failed to initialize and the parameters passed in were incorrect.

Code examples

```

struct i2s_device *i2s_test = NULL;

i2s_test = (struct i2s_device*)dev_get(HG_IIS0_DEVID);

/* Obtain the handle of I2S0 through dev_get(), and configure the host mode, the sampling rate is 44.1k,
The number of audio data bits is 16bit */

i2s_open(i2s_test, I2S_MODE_MASTER, I2S_SAMPLE_FREQ_44_1K, \
I2S_SAMPLE_BITS_16BITS);

```

3.4.2. i2s_close()

Turn off I2S. After calling this function, I2S will not work properly, and all configurations (including interrupt-related configurations) will fail.

Function prototype

```
int32 i2s_close(struct i2s_device *i2s)
```

Function parameters

parameter	type	illustrate
i2s	struct i2s_device	I2S handle. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	I2S module closed successfully
RET_ERR	int32	I2S module shutdown failed

Code examples

```
struct i2s_device *i2s_test = NULL;

i2s_test = (struct i2s_device*)dev_get(HG_IIS0_DEVID);

/* Get the handle of I2S0 through dev_get() and close I2S0*/

i2s_close(i2s_test);
```

3.4.3. i2s_write()

I2S sends data based on the address of the DATA BUFFER and the number of data. Users can complete the transmission in the I2S module. When the input is interrupted, switch the address and data number of the DATA BUFFER to be sent next time. The I2S module will complete this transmission. After completion, it will automatically switch to the next DATA BUFFER and data quantity. If the user is in the middle of the completed transmission of the I2S module. When interrupted, the address and length are not switched. The I2S module will reload this DATA BUFFER after this transmission is completed. and the number of data.

Note: The user can only switch the next address and data when the I2S module is interrupted in half of the transmission, otherwise it will cause Causes sending exception.

Function prototype

```
int32 i2s_write(struct i2s_device *i2s, const void* buf, uint32 len)
```

Function parameters

parameter	type	illustrate
i2s	struct i2s_device	I2S handle. Usually use dev_get() to get the handle
buf	const void*	Starting address of DATA BUFFER
only	uint32	The number of data to be sent, in byte, requires 4byte alignment

Return value

return value	type	illustrate
RET_OK	int32	The I2S module writes the BUFFER address and length successfully
RET_ERR	int32	I2S module failed to write BUFFER address and length

Code examples

```
uint16 data[256];

struct i2s_device *i2s_test = NULL;

/* Initialize data at will, just for example, no practical significance*/

memset((void *)data, 0x55, sizeof(data)/sizeof(data[0]));

i2s_test = (struct i2s_device*)dev_get(HG_IIS0_DEVID);

/* Obtain the handle of I2S0 through dev_get(), and configure the host mode, the sampling rate is 44.1k,
The number of audio data bits is 16bit */

i2s_open(i2s_test, I2S_MODE_MASTER, I2S_SAMPLE_FREQ_44_1K, \
I2S_SAMPLE_BITS_16BITS);

/* Send 256*2byte */

i2s_write(i2s_test, (void *)data, sizeof(data)/sizeof(data[0]));
```

3.4.4. i2s_read()

I2S receives data based on the address of the DATA BUFFER and the number of data. Users can complete the transmission in the I2S module

When the input is interrupted, the address and data number of the DATA BUFFER to be received next are switched. The I2S module will complete this transmission

After completion, it will automatically switch to the next DATA BUFFER and data quantity. If the user is in the middle of the completed transmission of the I2S module

When interrupted, the address and length are not switched. The I2S module will reload this DATA BUFFER after this transmission is completed.

and the number of data.

Note: The user can only switch the next address and data when the I2S module is interrupted in half of the transmission, otherwise it will cause

Causes sending exception.

Function prototype

```
int32 i2s_read(struct i2s_device *i2s, void* buf, uint32 len)
```

Function parameters

parameter	type	illustrate
i2s	struct i2s_device	I2S handle. Usually use dev_get() to get the handle
buf	void*	Starting address of DATA BUFFER

only	uint32	The number of data to be received, in byte, requires 4byte alignment
------	--------	--

Return value

return value	type	illustrate
RET_OK	int32	The I2S module writes the BUFFER address and length successfully
RET_ERR	int32	I2S module failed to write BUFFER address and length

Code examples

```
uint16 data[256];

struct i2s_device *i2s_test = NULL;

/* Initialize data at will, just for example, no practical significance*/
memset((void *)data, 0x00, sizeof(data)/sizeof(data[0]));

i2s_test = (struct i2s_device*)dev_get(HG_IIS0_DEVID);

/* Obtain the handle of I2S0 through dev_get(), and configure the host mode, the sampling rate is 44.1kHz,
The number of audio data bits is 16bit */

i2s_open(i2s_test, I2S_MODE_MASTER, I2S_SAMPLE_FREQ_44_1K, \
I2S_SAMPLE_BITS_16BITS);

/*Receive 256*2byte */

i2s_read(i2s_test, (void *)data, sizeof(data)/sizeof(data[0]));
```

3.4.5. i2s_ioctl()

According to the command in enum i2s_ioctl_cmd, call this function to configure the I2S module.

Function prototype

```
int32 i2s_ioctl( struct i2s_device *i2s,
                uint32 cmd,
                uint32 param,
                )
```

Function parameters

parameter	type	illustrate
i2s	struct i2s_device	I2S handle. Usually use dev_get() to get the handle
cmd	uint32	For configuration commands of the I2S module, see enumeration i2s_ioctl_cmd
param	uint32	Configuration parameters, depending on the configuration command

Return value

return value	type	illustrate
RET_OK	int32	I2S module configured successfully
RET_ERR	int32	I2S module configuration failed

Code examples

```
struct i2s_device *i2s_test = NULL;

i2s_test = (struct i2s_device*)dev_get(HG_IIS0_DEVID);

/* Obtain the handle of I2S0 through dev_get(), and configure the host mode, the sampling rate is 44.1kHz,
The number of audio data bits is 16bit */

i2s_open(i2s_test, I2S_MODE_MASTER, I2S_SAMPLE_FREQ_44_1K, \
I2S_SAMPLE_BITS_16BITS);

/* Configure the I2S channel mode to mono through i2s_ioctl */

i2s_ioctl(i2s_test,I2S_IOCTL_CMD_SET_CHANNEL, I2S_CHANNEL_MONO);
```

3.4.6. i2s_request_irq()

According to the interrupt type of enum i2s_irq_flag, call this function to apply for the interrupt of the I2S module.

Function prototype

```
int32 i2s_request_irq( struct i2s_device *i2s,
                      uint32 irq_flag,
                      i2s_irq_hdl irq_hdl,
```


3.4.7. i2s_release_irq()

According to the interrupt type of enum i2s_irq_flag, call this function to release the interrupt of the I2S module.

Function prototype

```
int32 i2s_release_irq(struct i2s_device *i2s, uint32 irq_flag)
```

Function parameters

parameter	type	illustrate
i2s	struct i2s_device	I2S handle. Usually use dev_get() to get the handle
irq_flag	uint32	Released interrupt type, refer to enumeration i2s_irq_flag

Return value

return value	type	illustrate
RET_OK	int32	I2S module interrupt closed successfully
RET_ERR	int32	I2S module interrupt shutdown failed

Code examples

```
struct i2s_device      *i2s_test;

i2s_test = (struct i2s_device*)dev_get(HG_IIS0_DEVID);

/* Obtain the handle of I2S0 through dev_get(), and configure the host mode, the sampling rate is 44.1kHz,
The number of audio data bits is 16bit */

i2s_open(i2s_test, I2S_MODE_MASTER, I2S_SAMPLE_FREQ_44_1K, \
I2S_SAMPLE_BITS_16BITS);

/* Close half-transmission interrupt */

i2s_release_irq(i2s_test, I2S_IRQ_FLAG_HALF);
```

4. PDM interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of PDM. They are located in the "" file directory

4.1. Enum

4.1.1. pdm_sample_freq

pdm_sample_freq describes the sampling frequency.

enumerator	illustrate
PDM_SAMPLE_FREQ_8K	PDM sampling frequency is 8KHz
PDM_SAMPLE_FREQ_16K	PDM sampling frequency is 16KHz
PDM_SAMPLE_FREQ_32K	PDM sampling frequency is 32KHz
PDM_SAMPLE_FREQ_48K	PDM sampling frequency is 48KHz

4.1.2. pdm_channel

pdm_channel describes the working mode selection.

enumerator	illustrate
PDM_CHANNEL_LEFT	PDM works in left channel mode
PDM_CHANNEL_RIGHT	PDM works in left channel mode
PDM_CHANNEL_STEREO	PDM works in stereo mode

4.1.3. pdm_irq_flag

pdm_irq_flag describes the interrupt types that PDM can apply for, through pdm_request_irq() and

pdm_release_irq() function is used.

enumerator	illustrate
PDM_IRQ_FLAG_DMA_HF	PDM half-completed transfer interrupted

PDM_IRQ_FLAG_DMA_OV	PDM completes all transfers interrupted
---------------------	---

4.1.4. pdm_ioctl_cmd

pdm_ioctl_cmd describes the PDM configuration command, which is configured by calling pdm_ioctl().

enumerator	illustrate
PDM_IOCTL_CMD_LR_CHANNEL_INTERCHANGE	PDM left and right channel swap

4.2. Define

4.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

4.3. Structure

4.3.1. pdm_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type	illustrate
dev	struct dev_obj Describes operating system related content
open	int32 function pointer
write	int32 function pointer
close	int32 function pointer

request_irq	int32 function pointer	
release_irq	int32 function pointer	
ioctl	int32 function pointer	

4.4. Function

4.4.1. pdm_open()

Initialize PDM. This function must be called before using the PDM module. After PDM initialization is successful, the default

The default configuration is: frequency reduction ratio (fs) = 64; the left and right channels are not interchanged. If you need to change the configuration, you can after the initialization is completed,

Use the pdm_ioctl function to make changes.

Function prototype

```
int32 pdm_open( struct pdm_device *pdm,
               enum pdm_sample_freq freq,
               enum pdm_channel channel
               )
```

Function parameters

parameter	type	illustrate
pdm	struct pdm_device	Handle to PDM. Usually use dev_get() to get the handle
freq	enum pdm_sample_freq	PDM sampling frequency, refer to enumeration pdm_sample_freq
channel	enum pdm_channel	PDM working mode, refer to enumeration pdm_channel

Return value

return value	type	illustrate
RET_OK	int32	PDM module initialization successful
RET_ERR	int32	PDM module initialization failed

Code examples

```
struct pdm_device *pdm_test = NULL;

pdm_test = (struct pdm_device*)dev_get(HG_PDM_DEVID);

/* Get the PDM handle through dev_get() and configure stereo sound with a sampling rate of 16k */

pdm_open(pdm_test, PDM_SAMPLE_FREQ_16K, PDM_CHANNEL_STEREO);
```

4.4.2. pdm_read()

Fill in the DATA BUFFER address and length, and enable PDM to start transmission. Users can complete the transmission in half of the PDM module

When interrupted, switch the address and data number of the DATA BUFFER to be received next. The PDM module will be completed during this transfer

After that, it will automatically switch to the next DATA BUFFER and data quantity. If the user interrupts the transmission halfway through the PDM module

When , the address and length are not switched. The PDM module will reload this DATA BUFFER after this transmission is completed.

and the number of data.

Note: The user can only switch the next address and data when the PDM module is interrupted halfway through the transmission, otherwise it will cause

Causes reception exception.

Function prototype

```
int32 pdm_read(struct pdm_device *pdm, void *buf, uint32 len)
```

Function parameters

parameter	type	illustrate
pdm	struct pdm_device	Handle to PDM. Usually use dev_get() to get the handle
buf	void*	Starting address of DATA BUFFER
only	uint32	The number of data to be received, in byte

Return value

return value	type	illustrate
RET_OK	int32	PDM module writes BUFFER address and length successfully
RET_ERR	int32	PDM module writes BUFFER address and length successfully

Code examples

```
uint16 data[256];

struct pdm_device *pdm_test = NULL;

/* Initialize data at will, just for example, no practical significance*/

memset((void *)data, 0x00, sizeof(data)/sizeof(data[0]));

pdm_test = (struct pdm_device*)dev_get(HG_PDM_DEVID);

/* Get the PDM handle through dev_get() and configure stereo sound with a sampling rate of 16k */

pdm_open(pdm_test, PDM_SAMPLE_FREQ_16K, PDM_CHANNEL_STEREO);

/*Receive 256*2byte */

pdm_read(pdm_test, (void *)data, sizeof(data)/sizeof(data[0]));
```

4.4.3. pdm_close()

Close PDM. After calling this function, PDM will not work properly and needs to be opened again using the pdm_open() function.

Function prototype

```
int32 pdm_close(struct pdm_device *pdm)
```

Function parameters

parameter	type	illustrate
pdm	struct pdm_device	Handle to PDM. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	PDM module closed successfully
RET_ERR	int32	PDM module shutdown failed

Code examples

```
struct pdm_device *pdm_test = NULL;

pdm_test = (struct pdm_device*)dev_get(HG_PDM_DEVID);

/* Obtain the handle of PDM through dev_get() and close PDM*/

pdm_close(pdm_test);
```

4.4.4. pdm_request_irq()

According to the interrupt type of enum pdm_irq_flag, call this function to apply for the interrupt of the PDM module.

Function prototype

```
int32 pdm_request_irq( struct pdm_device *pdm,

                      enum pdm_irq_flag flag,

                      pdm_irq_hdl          irq_hdl,

                      uint32                data

                      )
```

Function parameters

parameter	type	illustrate
pdm	struct pdm_device	Handle to PDM. Usually use dev_get() to get the handle
flag	enum pdm_irq_flag	The requested interrupt type, refer to enumeration pdm_irq_flag
irq_hdl	pdm_irq_hdl	Interrupt handle, executed after the interrupt is generated
data	uint32	Interrupt handle parameters

Return value

return value	type	illustrate
RET_OK	int32	PDM module applied for interruption successfully
RET_ERR	int32	PDM module failed to apply for interrupt

Code examples

```
void pdm_interrupt_func(uint32 irq, uint32 irq_data) {
    __NOP();
}

struct pdm_device *pdm_test = NULL;

pdm_test = (struct pdm_device*)dev_get(HG_PDM_DEVID);

/* Get the PDM handle through dev_get() and configure stereo sound with a sampling rate of 16k */

pdm_open(pdm_test, PDM_SAMPLE_FREQ_16K, PDM_CHANNEL_STEREO);

/*Apply for the half-completed transmission interrupt of the PDM module*/

pdm_request_irq(pdm_test, PDM_IRQ_FLAG_DMA_HF, pdm_interrupt_func, 0);
```

4.4.5. pdm_release_irq()

According to the interrupt type of enum pdm_irq_flag, call this function to release the interrupt of the PDM module.

Function prototype

```
int32 pdm_release_irq(struct pdm_device *pdm, enum pdm_irq_flag irq_flag)
```

Function parameters

parameter	type	illustrate
pdm	struct pdm_device	Handle to PDM. Usually use dev_get() to get the handle
irq_flag	enum pdm_irq_flag	Released interrupt type, refer to enumeration pdm_irq_flag

Return value

return value	type	illustrate
RET_OK	int32	PDM module interrupt closed successfully
RET_ERR	int32	PDM module interrupt shutdown failed

Code examples

```
struct pdm_device *pdm_test = NULL;

pdm_test = (struct pdm_device*)dev_get(HG_PDM_DEVID);

/* Get the PDM handle through dev_get() and configure stereo sound with a sampling rate of 16k */

pdm_open(pdm_test, PDM_SAMPLE_FREQ_16K, PDM_CHANNEL_STEREO);

/* Turn off the half-completed transmission interrupt of the PDM module */

pdm_release_irq(pdm_test, PDM_IRQ_FLAG_DMA_HF);
```

4.4.6. pdm_ioctl()

According to the command in enum pdm_ioctl_cmd, call this function to configure the PDM module.

Function prototype

```
int32 pdm_ioctl(struct pdm_device *pdm,
               enum pdm_ioctl_cmd cmd,
               uint32 param
               )
```

Function parameters

parameter	type	illustrate
pdm	struct pdm_device	Handle to PDM. Usually use dev_get() to get the handle
cmd	enum pdm_ioctl_cmd	For configuration commands of the PDM module, see enumeration pdm_ioctl_cmd
param	uint32	Configuration parameters, depending on the configuration command

Return value

return value	type	illustrate
RET_OK	int32	PDM module configured successfully
RET_ERR	int32	PDM module configuration failed

Code examples

```

struct pdm_device *pdm_test = NULL;

pdm_test = (struct pdm_device*)dev_get(HG_PDM_DEVID);

/* Get the PDM handle through dev_get() and configure stereo sound with a sampling rate of 16k */

pdm_open(pdm_test, PDM_SAMPLE_FREQ_16K, PDM_CHANNEL_STEREO);

/* Configure left and right channel swap*/

pdm_ioctl(pdm_test, PDM_IOCTL_CMD_LR_CHANNEL_INTERCHANGE, 1);

```

5. ADC interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of ADC. They are located in the "" file directory

5.1. Enum

5.1.1. adc_irq_flag

adc_irq_flag describes the interrupt types that ADC can apply for, through adc_request_irq() and adc_release_irq() function is used.

enumerator	illustrate
ADC_IRQ_FLAG_SAMPLE_DONE	ADC sampling complete interrupt

5.1.2. adc_ioctl_cmd

adc_ioctl_cmd describes the configuration command of ADC, which is configured by calling adc_ioctl().

enumerator	illustrate
RESERVE	Reserved for subsequent use

5.1.3. adc_voltage_type

adc_voltage_type describes the type of chip internal voltage that the ADC can collect, by calling adc_add_channel() adds a sampling channel.

Note:

1. In addition to sampling the internal voltage of the chip, the ADC also supports adding a channel for sampling the I/O voltage. The channel name is PA_0-PA_15, PB_0-PB_15, PC_0-PC_5 enumeration values.
2. The enumeration value of the chip's internal voltage type in adc_voltage_type.h must start from 0x101.
3. The enumerator is located in "...sdk\include\chip\txw80x\adc_voltage_type.h".

enumerator	illustrate
ADC_CHANNEL_RF_TEMPERATURE	ADC sampling RF temperature path

5.2. Define

5.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

5.3. Structure

5.3.1. adc_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
open	int32 function pointer	
close	int32 function pointer	
add_channel	int32 function pointer	

delete_channel	int32 function pointer	
get_value	int32 function pointer	
ioctl	int32 function pointer	
request_irq	int32 function pointer	
release_irq	int32 function pointer	

5.4. Function

5.4.1. adc_open()

Initialize ADC. This function must be called before using the ADC module.

Function prototype

```
int32 adc_open(struct adc_device *adc)
```

Function parameters

parameter	type	illustrate
adc	struct adc_device	ADC handle. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	ADC module initialization successful
RET_ERR	int32	ADC module initialization failed

Code examples

```

struct adc_device *adc_test = NULL;

adc_test = (struct adc_device*)dev_get(HG_ADC_DEVID);

/* Get the ADC handle through dev_get() and open the ADC */

adc_open(adc_test);

```

5.4.2. adc_close()

Turn off the ADC. After calling this function, the ADC will not work properly and needs to be opened again using the `adc_open()` function.

Function prototype

```
int32 adc_close(struct adc_device *adc)
```

Function parameters

parameter	type	illustrate
adc	struct adc_device	ADC handle. Usually use <code>dev_get()</code> to get the handle

Return value

return value	type	illustrate
RET_OK	int32	ADC module closed successfully
RET_ERR	int32	ADC module shutdown failed

Code examples

```

struct adc_device *adc_test = NULL;

adc_test = (struct adc_device*)dev_get(HG_ADC_DEVID);

/* Get the ADC handle through dev_get() and open the ADC */

adc_open(adc_test);

/* Obtain the ADC handle through dev_get() and close the ADC */

adc_close(adc_test);

```

5.4.3. adc_add_channel()

Add ADC sampling path. The sampling channel of ADC supports chip internal voltage and I/O voltage. For details, see enum `adc_voltage_type`

Note:

- 1. Unable to add duplicate sampling channels.

Function prototype

```
int32 adc_add_channel(struct adc_device *adc, uint32 channel)
```

Function parameters

parameter	type	illustrate
adc	struct adc_device	ADC handle. Usually use <code>dev_get()</code> to get the handle
channel	uint32	ADC sampling channel, see enumeration <code>adc_voltage_type</code> for details

Return value

return value	type	illustrate
RET_OK	int32	ADC module adds sampling channel successfully
RET_ERR	int32	ADC module failed to add sampling channel

Code examples

```
struct adc_device *adc_test = NULL;

adc_test = (struct adc_device*)dev_get(HG_ADC_DEVID);

/* Get the ADC handle through dev_get() and open the ADC */

adc_open(adc_test);

/* Add a channel for sampling PA_0 voltage */

adc_add_channel(adc_test, PA_0);

/* Add the RF temperature path inside the sampling chip*/

adc_add_channel(adc_test, ADC_CHANNEL_RF_TEMPERATURE);
```

5.4.4. adc_delete_channel()

Delete the added ADC sampling channel. After deleting a channel, the channel will not be able to perform ADC sampling.

Note:

- 1. Cannot delete non-existing channels

Function prototype

```
int32 adc_delete_channel(struct adc_device *adc, uint32 channel)
```

Function parameters

parameter	type	illustrate
adc	struct adc_device	ADC handle. Usually use dev_get() to get the handle
channel	uint32	ADC sampling channel, see enumeration adc_voltage_type for details

Return value

return value	type	illustrate
RET_OK	int32	ADC module deletes sampling channel successfully
RET_ERR	int32	ADC module failed to delete sampling channel

Code examples

```
struct adc_device *adc_test = NULL;

adc_test = (struct adc_device*)dev_get(HG_ADC_DEVID);

/* Get the ADC handle through dev_get() and open the ADC */
adc_open(adc_test);

/* Add a channel for sampling PA_0 voltage */
adc_add_channel(adc_test, PA_0);

/* Add the RF temperature path inside the sampling chip*/
adc_add_channel(adc_test, ADC_CHANNEL_RF_TEMPERATURE);

/* Delete the added channel for sampling PA_0 voltage */
adc_delete_channel(adc_test, PA_0);
```

5.4.5. adc_get_value()

Get the ADC sampling value of the current sampling channel.

Note:

1. ADC has a 12-bit conversion accuracy, so the maximum ADC sampling value is $2^{12} = 4095$.
2. The voltage of unadded sampling channels cannot be collected.
3. After obtaining the ADC sampling value through this function, calculate the current voltage value through the formula, as follows:

ADC sampling value=2048; reference voltage=3.3V;

Voltage value = $(2048/4095)*3.3 = 1.65V$

4. If the sampling channel is ADC_CHANNEL_RF_TEMPERATURE, the value obtained is the current temperature value.

Not the ADC sample value.

Function prototype

```
int32 adc_get_value(struct adc_device *adc,
                   uint32 channel,
                   uint32 *raw_data)
{
    ...
}
```

Function parameters

parameter	type	illustrate
adc	struct adc_device	ADC handle. Usually use dev_get() to get the handle
channel	uint32	ADC sampling channel, see enumeration adc_voltage_type for details
raw_data	uint32*	After ADC sampling is completed, the sample value returned

Return value

return value	type	illustrate
RET_OK	int32	ADC module sampled successfully
RET_ERR	int32	ADC module sampling failed

Code examples

```

struct adc_device *adc_test = NULL;

uint32 adc_raw_data = 0;

uint32 voltage = 0;

adc_test = (struct adc_device*)dev_get(HG_ADC_DEVID);

/* Get the ADC handle through dev_get() and open the ADC */

adc_open(adc_test);

/* Add a channel for sampling PA_0 voltage */

adc_add_channel(adc_test, PA_0);

/* Get the voltage of PA_0 and return the ADC sampling value through adc_raw_data */

adc_get_value(adc_test, PA_0, &adc_raw_data);

/* Calculate the voltage value on PA_0 at this time*/

voltage = (adc_raw_data / 4095) * 3.3;

```

5.4.6. adc_ioclt()

According to the command in enum adc_ioclt_cmd, call this function to configure the ADC module.

Function prototype

```

int32 adc_ioclt(struct adc_device *adc,

               enum adc_ioclt_cmd ioclt_cmd,

               uint32 param1,

               uint32 param2

               )

```

Function parameters

parameter	type	illustrate
adc	struct adc_device	ADC handle. Usually use dev_get() to get the handle
ioclt_cmd	enum adc_ioclt_cmd	For configuration commands of the ADC module, see enumeration adc_ioclt_cmd
param1	uint32	Configuration parameters, depending on the configuration command
param2	uint32	Configuration parameters, depending on the configuration command

Return value

return value	type	illustrate
RET_OK	int32	ADC module configured successfully
RET_ERR	int32	ADC module configuration failed

Code examples

```
/* The ADC module has not yet set ioctl_cmd, so there is no routine */
```

5.4.7. adc_request_irq()

According to the interrupt type of enum adc_irq_flag, call this function to apply for the interrupt of the ADC module.

Function prototype

```
int32 adc_request_irq(struct adc_device *adc,
                    enum adc_irq_flag irq_flag,
                    adc_irq_hdl          irq_hdl,
                    uint32                irq_data
                    )
```

Function parameters

parameter	type	illustrate
adc	struct adc_device	ADC handle. Usually use dev_get() to get the handle
irq_flag	enum adc_irq_flag	The interrupt type applied for, refer to the enumeration adc_irq_flag
irq_hdl	adc_irq_hdl	Interrupt handle, executed after the interrupt is generated
irq_data	uint32	Interrupt handle parameters

Return value

return value	type	illustrate
RET_OK	int32	ADC module applied for interrupt successfully
RET_ERR	int32	ADC module failed to apply for interrupt

Code examples

```
void adc_interrupt_func(uint32 irq, uint32 channel, uint32 irq_data) {
    __NOP();
}

struct adc_device *adc_test = NULL;

adc_test = (struct adc_device*)dev_get(HG_ADC_DEVID);

/* Get the ADC handle through dev_get() and open the ADC */

adc_open(adc_test);

/* Add a channel for sampling PA_0 voltage */

adc_add_channel(adc_test, PA_0);

/* Apply for the sampling completion interrupt of the ADC module*/

adc_request_irq(adc_test, ADC_IRQ_FLAG_SAMPLE_DONE, \
adc_interrupt_func, 0);
```

5.4.8. adc_release_irq()

According to the interrupt type of enum adc_irq_flag, call this function to release the interrupt of the ADC module.

Function prototype

```
int32 adc_release_irq(struct adc_device *adc, enum adc_irq_flag irq_flag)
```

Function parameters

parameter	type	illustrate
adc	struct adc_device	ADC handle. Usually use dev_get() to get the handle
irq_flag	enum adc_irq_flag	The type of interrupt released, refer to enumeration adc_irq_flag

Return value

return value	type	illustrate
RET_OK	int32	ADC module interrupt is closed successfully
RET_ERR	int32	ADC module interrupt shutdown failed

Code examples

```
struct adc_device *adc_test = NULL;

adc_test = (struct adc_device*)dev_get(HG_ADC_DEVID);

/* Get the ADC handle through dev_get() and open the ADC */

adc_open(adc_test);

/* Add a channel for sampling PA_0 voltage */

adc_add_channel(adc_test, PA_0);

/* Turn off the sampling completion interrupt of the ADC module */

adc_release_irq(adc_test, ADC_IRQ_FLAG_SAMPLE_DONE);
```

6. DVP interface description

This chapter mainly introduces the functions of DVP interface functions, enumerations, macros, and structures. It is located in the "" file directory.

6.1. Enum

6.1.1. dvp_ioctl_cmd

dvp_ioctl_cmd describes the ioctl configuration command of the DVP interface, using the ioctl in the dvp_device structure

accomplish.

enumerator	illustrate
DVP_IOCTL_CMD_SET_FORMAT	Set the input format of DVP (YUV, RGB, JPG)
DVP_IOCTL_CMD_RGB_2_YUV	When setting RGB input, convert RGB data to YUV
DVP_IOCTL_CMD_SET_SIZE	Set the size of the image input
DVP_IOCTL_CMD_SET_ADR_1	Set the data receiving address BUF1 of DVP
DVP_IOCTL_CMD_SET_ADR_2	Set the data receiving address BUF2 of DVP
DVP_IOCTL_CMD_SET_SCEN	Set the image data to be reduced, and the length and width will be doubled.
DVP_IOCTL_CMD_SET_HSYNC_VAILD	Effective level configuration of HSYNC
DVP_IOCTL_CMD_SET_VSYNC_VAILD	Effective level configuration of VSYNC
DVP_IOCTL_CMD_SET_ONE_SAMPLE	Set a single image output. After this configuration is turned on, the dvp image will only be captured once.
DVP_IOCTL_CMD_SET_DEBOUNCE	PCLK filtering, configure filter period
DVP_IOCTL_CMD_SET_YCBCR_MODE	In YUV format, the brightness and chroma arrangement order of the input source
DVP_IOCTL_CMD_DIS_UV_MODE	In YUV format, start capturing only brightness (Y) data.
DVP_IOCTL_CMD_SET_FRAME_RATE	Set frame rate control, 25%/50%/75%/100% capture efficiency
DVP_IOCTL_CMD_SET_THRESHOLD	Set the upper and lower range of Y, UV data (less than the lowest value, greater than the maximum value)
DVP_IOCTL_CMD_EX_D5_D6	Whether the D5 and D6 data pins of DVP need to be exchanged depends on the hardware wiring.

DVP_IOCTL_CMD_SET_JPEG_LEN	When the input format is configured as JPG, set the data length for BUF data full interrupt
----------------------------	---

6.2. Define

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

6.3. Structure

6.3.1. dvp_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
heat	int32 function pointer	Enable dvp clock
baudrate	int32 function pointer	Mclk frequency configuration
open	int32 function pointer	Dvp function turned on
close	int32 function pointer	Dvp function is turned off
ioctl	int32 function pointer	Dvp configuration information, function call
Request_irq	int32 function pointer	Interrupt application
Release_irq	int32 function pointer	Interrupt release

6.4. Function

6.4.1. dvp_init()

dvp pre-start configuration, clock on

Function prototype

```
int32 dvp_init(struct dvp_device *p_dvp)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of dvp. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	dvp module initialization successful
RET_ERR	int32	dvp module initialization failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Obtain the handle of dvp through dev_get() and initialize dvp
*/

dvp_init(dvp_test);
```

6.4.2. dvp_open()

dvp function activated

Function prototype

```
int32 dvp_open(struct dvp_device *p_dvp)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of dvp. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	dvp module opened successfully
RET_ERR	int32	dvp module failed to open

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Get the handle of dvp through dev_get() and open the dvp module
*/

dvp_open(dvp_test);
```

6.4.3. dvp_close()

dvp function is turned off

Function prototype

```
int32 dvp_close(struct dvp_device *p_dvp)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of dvp. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	dvp module closed successfully

RET_ERR	int32	dvp module shutdown failed
---------	-------	----------------------------

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Obtain the handle of dvp through dev_get() and close the dvp module
*/

dvp_close(dvp_test);
```

6.4.4. dvp_set_baudrate()

Configure dvp output clock frequency

Function prototype

```
int32 dvp_set_baudrate(struct dvp_device *p_dvp,uint32 mclk)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
mclk	uint32	DVP output mclk clock frequency

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Obtain the handle of dvp through dev_get(), configure the mclk clock output of dvp, the clock is 24M
*/

dvp_set_baudrate(dvp_test,24000000);
```

6.4.5. dvp_set_size()

Configure dvp Configure image display coordinates

Function prototype

```
int32 dvp_set_size(struct dvp_device *p_dvp, uint32 x_s, uint32 y_s, uint32
x_e, uint32 y_e)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
x_s	uint32	Image starting abscissa
y_s	uint32	Image starting ordinate
x_e	uint32	The end abscissa of the image is -1
y_e	uint32	The vertical coordinate of the end of the image is -1

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples


```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*

Obtain the handle of dvp through dev_get(), configure the resolution of the input image, and set the horizontal

The coordinate address is 0 to 640-1, and the vertical coordinate address is 0 to 480-1

*/

dvp_set_size(dvp_test,0,0,640-1,480-1);
```

6.4.6. dvp_set_addr1()/dvp_set_addr2()

Configure dvp data cache location

Function prototype

```
int32 dvp_set_addr1(struct dvp_device *p_dvp, uint32 yuv_addr)

int32 dvp_set_addr2(struct dvp_device *p_dvp, uint32 yuv_addr)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
yuv_addr	uint32	Buf space address

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

uint8 yuvbuf[2][IMAGE_W*8*2+IMAGE_W*8+16];

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Get the handle of dvp through dev_get() and set the buf address of dvp
*/

dvp_set_addr1(dvp_test,ybuf[0]);

dvp_set_addr2(dvp_test,ybuf[1]);
```

6.4.7. dvp_set_rgb2yuv()

Configure dvp data to convert from rgb565 to yuv

Function prototype

```
int32 dvp_set_rgb2yuv(struct dvp_device *p_dvp, uint8en)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
in	uint8	Whether to start rgb565 to yuv conversion

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Obtain the handle of dvp through dev_get() and enable the input source to be converted from rgb565 to yuv data.
*/

dvp_set_rgb2yuv(dvp_test,1);
```

6.4.8. dvp_set_format()

Configure dvp input source configuration

Function prototype

```
int32 dvp_set_format(struct dvp_device *p_dvp, uint8format)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
format	uint8	DVP input source format 0: YES 1:RGB565 2:JPEG

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*

Obtain the handle of dvp through dev_get(), and configure the dvp input source image format to be YUV

*/

dvp_set_format(dvp_test,0);
```

6.4.9. dvp_set_half_size()

Configure dvp to double the length and width

Function prototype

```
int32 dvp_set_half_size(struct dvp_device *p_dvp, uint8en)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
in	uint8	DVP output length and width are reduced by half

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*

Obtain the handle of dvp through dev_get(), configure the resolution reduction function of the input image, and reduce the length and width by half.

That is, VGA changes to QVGA

*/

dvp_set_half_size(dvp_test,1);
```

6.4.10. dvp_set_vsync_polarity()/dvp_set_hsync_polarity()

Configure dvp input vs, hs effective level configuration

Function prototype

```
int32 dvp_set_vsync_polarity(struct dvp_device *p_dvp, uint8 high_valid)

int32 dvp_set_hsync_polarity(struct dvp_device *p_dvp, uint8 high_valid)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
high_valid	uint8	The effective level setting of Hsync and Vsync of DVP, 1 is high effective

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Obtain the handle of dvp through dev_get() and configure the effective level of vsync/hsync to high level
*/

dvp_set_vsync_polarity(dvp_test,1);

dvp_set_hsync_polarity(dvp_test,1);
```

6.4.11. dvp_set_once_sampling()

Configure single capture for dvp

Function prototype

```
int32 dvp_set_once_sampling(struct dvp_device *p_dvp, uint8en)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
in	uint8	DVP starts a single capture, and the module will only capture images once

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*

Obtain the handle of dvp through dev_get(), and configure to only capture one frame of image.

*/

dvp_set_once_sampling(dvp_test,1);
```

6.4.12. dvp_debounce_enable()

Configure the pclk filtering function of dvp

Function prototype

```
int32 dvp_debounce_enable(struct dvp_device *p_dvp, uint8enuint8pixel)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
in	uint8	DVP filter function enabled
pixel	uint8	filter period

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*

Obtain the handle of dvp through dev_get(), enable pclk filtering of dvp, and the filtering period is 2

*/

dvp_debounce_enable(dvp_test,1,2);
```

6.4.13. dvp_set_ycbcr()

When the input format is YUV, configure the YUV data layout of dvp

Function prototype

```
int32 dvp_set_ycbcr(struct dvp_device *p_dvp, uint8mode)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
mode	uint8	YUV data layout 0:YUYV 1:YVYU 2: UYVY 3:VYUY

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples


```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Obtain the handle of dvp through dev_get(). When the input source is yuv, the input format of yuv is yuyv.
*/

dvp_set_ycbcr(dvp_test,0);
```

6.4.14. dvp_unload_uv()

Configure pure brightness collection of dvp

Function prototype

```
int32 dvp_unload_uv(struct dvp_device *p_dvp, uint8en)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
in	uint8	Enable pure brightness collection, DVP invalidates the collection of color data

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*

Obtain the handle of dvp through dev_get() and configure to only collect brightness data.

*/

dvp_unload_uv(dvp_test,1);
```

6.4.15. dvp_frame_load_precent()

Configure the frame sampling frequency of dvp

Function prototype

```
int32 dvp_frame_load_precent(struct dvp_device *p_dvp, uint8precent)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
percent	uint8	DVP frame sampling frequency 0:100% 1:75% 2:50% 3:25%

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*

Obtain the handle of dvp through dev_get(), configure the sampling rate to 50%, and only take one of every two pictures.

*/

dvp_frame_load_precent(dvp_test,2);
```

6.4.16. dvp_low_high_threshold()

Configure the yuv high and low thresholds of dvp

Function prototype

```
int32 dvp_low_high_threshold(struct dvp_device *p_dvp,bool low_high)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
low_high	bool	DVP enables high and low threshold functions, and the threshold range can be configured in the function Byte[0]:Y value Byte[1]:U value Byte[2]:V value

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Obtain the handle of dvp through dev_get() and enable the yuv data threshold function
*/

dvp_low_high_threshold(dvp_test,1);
```

6.4.17. dvp_set_exchange_d5_d6()

Configure the d5 and d6 pin function exchange of dvp

Function prototype

```
int32 dvp_set_exchange_d5_d6(struct dvp_device *p_dvp, uint8en)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
in	uint8	Enable the function exchange between d5 and d6, which is related to the wiring of the chip

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Get the dvp handle and dvp IO pin through dev_get(), when IO5->PB11, IO6->PB10, d5/d6
The pins do not need to be swapped. When IO5->PB10, IO6->PB11, the pin swap function is enabled.
*/

dvp_set_exchange_d5_d6(dvp_test,1);
```

6.4.18. dvp_jpeg_mode_set_len()

When the input format of dvp is jpeg, configure the buf length of jpeg

Function prototype

```
int32 dvp_jpeg_mode_set_len(struct dvp_device *p_dvp, uint32len)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
only	uint32	The buf length of Jpeg. After enough buf length is taken, a buf full interrupt will be generated.

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Obtain the handle of dvp through dev_get(). When the dvp input is jpg, it is generated when the configured receiving data is 2048.
A buf full interrupt
*/

dvp_jpeg_mode_set_len(dvp_test,2048);
```

6.4.19. dvp_request_irq()

dvp interrupt registration function

Function prototype

```
int dvp_request_irq(struct dvp_device *p_dvp,
                    uint32 irq_flag,
                    dvp_irq_hdl irq_hdl,
                    uint32 irq_data
                    )
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
irq_flag	uint32	Subinterrupt number: 0:hsync interrupt 1:vsync interrupt 2: Frame completion interrupt, when the input source is jpeg, it is buf full interrupt. 3: FIFO half full interrupt 4: FIFO overflow interrupt 5:SYNC failure interrupt 6:JPEG completion interrupt

irq_hdl	dvp_irq_hdl	Registered interrupt execution function
irq_data	uint32	Interrupt execution function parameters

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```
struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Obtain the handle of dvp through dev_get(), and execute the interrupt processing function after receiving the vsync valid signal.
dvp_vsie_isr, the parameter passed is dvp_test
*/

dvp_request_irq (dvp_test,1, (dvp_irq_hdl )&dvp_vsie_isr,dvp_test);
```

6.4.20. dvp_release_irq()

When the input format of dvp is jpeg, configure the buf length of jpeg

Function prototype

```
int32 dvp_release_irq(struct dvp_device *p_dvp, uint32irq_flag)
```

Function parameters

parameter	type	illustrate
p_dvp	struct dvp_device	The handle of the DVP. Usually use dev_get() to get the handle
irq_flag	uint32	Release interrupt number

Return value

return value	type	illustrate
RET_OK	int32	dvp module configured successfully
RET_ERR	int32	dvp module configuration failed

Code examples

```

struct dvp_device *dvp_test = NULL;

dvp_test = (struct dvp_device*)dev_get(HG_DVP_DEVID);

/*
Obtain the handle of dvp through dev_get() and cancel the interrupt of vsync
*/

dvp_release_irq (dvp_test,1);

```

7. JPG interface description

This chapter mainly introduces the functions of JPG interface functions, enumerations, macros, and structures. It is located in the "" file directory.

7.1. Enum

7.1.1. jpg_ioctl_cmd

jpg_ioctl_cmd describes the ioctl configuration command of the JPG interface, using the ioctl in the jpg_device structure accomplish.

enumerator	illustrate
JPG_IOCTL_CMD_SET_ADR	Set the data address of JPG

JPG_IOCTL_CMD_SET_QT	Set fine-tuning of QT table, each quality table can be subdivided into 16 levels
JPG_IOCTL_CMD_SET_SIZE	Set input resolution for JPEG
JPG_IOCTL_CMD_UPDATE_QT	Replace quality table

7.2. Define

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

7.3. Structure

7.3.1. jpg_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
heat	int32 function pointer	Turn on the jpg clock and initialize each table
open	int32 function pointer	Turn on jpg function
close	int32 function pointer	Turn off jpg function
ioctl	int32 function pointer	Jpeg configuration information, function call
request_irq	int32 function pointer	Interrupt application

release_irq	int32 function pointer	Interrupt release
-------------	------------------------	-------------------

7.4. Function

7.4.1. jpg_init()

jpg initialization, jpg clock on, Huffman table configuration, initial quantization table configuration

Function prototype

```
int32 jpg_init(struct jpg_device *p_jpg , uint32 table_idx, uint32 qt)
```

Function parameters

parameter	type	illustrate
p_jpg	struct jpg_device	The handle of the JPG. Usually use dev_get() to get the handle
table_idx uint32		Which quantization table to use, there are 6 quality_tables by default
qt	uint32	Fine-tune the quantization table used, with 16 levels of fine-tuning

Return value

return value	type	illustrate
RET_OK	int32	jpg module initialized successfully
RET_ERR	int32	jpg module initialization failed

Code examples

```
struct jpg_device *jpg_test = NULL;

jpg_test = (struct jpg_device*)dev_get(HG_JPG_DEVID);

/*

Get the jpg handle through dev_get(), open the jpg, and use the second default quality_table table.

Adjust to the 4th level, level 8 means no change in quality

*/

jpg_init(jpg_test,2,4);
```

7.4.2. jpg_open()

jpg module function is turned on

Function prototype

```
int32 jpg_open(struct jpg_device *p_jpg)
```

Function parameters

parameter	type	illustrate
p_jpg	struct jpg_device	The handle of the JPG. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	jpg module opened successfully
RET_ERR	int32	jpg module failed to open

Code examples

```
struct jpg_device *jpg_test = NULL;

jpg_test = (struct jpg_device*)dev_get(HG_JPG_DEVID);

jpg_init(jpg_test,2,4)

//

//

//

/*

Obtain the handle of jpg through dev_get(), initialize jpg, and configure the corresponding function, then open the jpg function

able

*/

jpg_open(jpg_test);
```

7.4.3. jpg_close()

jpg module function is turned off

Function prototype

```
int32 jpg_close(struct jpg_device *p_jpg)
```

Function parameters

parameter	type	illustrate
p_jpg	struct jpg_device	The handle of the JPG. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	jpg module closed successfully
RET_ERR	int32	jpg module failed to close

Code examples

```
struct jpg_device *jpg_test = NULL;

jpg_test = (struct jpg_device*)dev_get(HG_JPG_DEVID);

/*
Get the jpg handle through dev_get() and turn off the jpg function
*/

jpg_close(jpg_test);
```

7.4.4. jpg_updata_dqt()

jpg module quantization table update

Function prototype

```
int32 jpg_updata_dqt(struct jpg_device *p_jpg,uint32 *dqtbuf)
```

Function parameters

parameter	type	illustrate
p_jpg	struct jpg_device	The handle of the JPG. Usually use dev_get() to get the handle
dqtbuf	uint32 *	Quantization table to update

Return value

return value	type	illustrate
RET_OK	int32	The jpg module updated the quantization table successfully
RET_ERR	int32	jpg module failed to update quantization table

Code examples

```
struct jpg_device *jpg_test = NULL;

extern char quality_tab[6][128];

jpg_test = (struct jpg_device*)dev_get(HG_JPG_DEVID);

/*

Obtain the handle of jpg through dev_get() and update the 0th quantization in the 6 quantization tables of quality_tab.

surface

*/

jpg_updata_dqt(jpg_test, quality_tab[0]);
```

7.4.5. jpg_set_qt()

jpg module quantization table update

Function prototype

```
int32 jpg_set_qt(struct jpg_device *p_jpg,uint32 qt)
```

Function parameters

parameter	type	illustrate
p_jpg	struct jpg_device	The handle of the JPG. Usually use dev_get() to get the handle
qt	uint32	Fine-tuning level, 0~15, 8 means the quantization table remains unchanged, upward decreases the compression level, downward Increase compression level

Return value

return value	type	illustrate
RET_OK	int32	The jpg module fine-tuned the quantization table successfully
RET_ERR	int32	jpg module failed to fine-tune quantization table

Code examples

```
struct jpg_device *jpg_test = NULL;

extern char quality_tab[6][128];

jpg_test = (struct jpg_device*)dev_get(HG_JPG_DEVID);

jpg_updata_dqt(jpg_test, quality_tab[0]);

/*

Get the handle of jpg through dev_get(), and fine-tune the quality of the currently selected quantization table No. 0 to improve

A class of image quality

*/

jpg_set_qt(jpg_test,7);
```

7.4.6. jpg_set_size()

jpg module quantization table update

Function prototype

```
int32 jpg_set_qt(struct jpg_device *p_jpg,uint32 h,uint32 w)
```

Function parameters

parameter	type	illustrate
p_jpg	struct jpg_device	The handle of the JPG. Usually use dev_get() to get the handle
h	uint32	input image height
ln	uint32	Input image width

return value	type	illustrate
RET_OK	int32	jpg module size set successfully
RET_ERR	int32	jpg module failed to set size

Code examples

```

struct jpg_device *jpg_test = NULL;

jpg_test = (struct jpg_device*)dev_get(HG_JPG_DEVID);

/*
Obtain the jpg handle through dev_get() and configure the input source resolution of the jpg module to VGA
*/

jpg_set_size(jpg_test,480,640);

```

7.4.7. jpg_set_addr()

jpg module quantization table update

Function prototype

```
int32 jpg_set_addr(struct jpg_device *p_jpg,uint32 addr,uint32 buflen)
```

Function parameters

parameter	type	illustrate
p_jpg	struct jpg_device	The handle of the JPG. Usually use dev_get() to get the handle
addr	uint32	The storage location of the data generated by the Jpg module
buflen	uint32	Data volume configuration for Jpg module data full interrupt

Return value

return value	type	illustrate
RET_OK	int32	jpg module set successfully
RET_ERR	int32	jpg module setup failed

Code examples

```
struct jpg_device *jpg_test = NULL;

uint8 jpgbuf[2048];

jpg_test = (struct jpg_device*)dev_get(HG_JPG_DEVID);

/*

Obtain the handle of jpg through dev_get(), configure the output buf position of the jpg module to jpgbuf, and the data is full

The interrupt length is 2048

*/

jpg_set_size(jpg_test, jpgbuf ,2048);
```

7.4.8. jpg_request_irq()

jpg module quantization table update

Function prototype

```
int32 jpg_request_irq(struct jpg_device *p_jpg,
                    jpg_irq_hdlirq_hdl,
                    uint32 irq_flags,
                    void *irq_data
                    )
```

Function parameters

parameter	type	illustrate
-----------	------	------------

p_jpg	struct jpg_device	The handle of the JPG. Usually use dev_get() to get the handle
irq_hdl	jpg_irq_hdl	Jpg corresponding interrupt processing function
irq_flags uint32		Jpg module subinterrupt number BIT (0): frame completion interrupt BIT(1): Frame buf data full interrupt BIT(2): Frame error interrupt
irq_data	void *	Parameters passed to the interrupt handler function

Return value

return value	type	illustrate
RET_OK	int32	jpg module subinterrupt registration successful
RET_ERR	int32	jpg module subinterrupt registration failed

Code examples

```

struct jpg_device *jpg_test = NULL;

uint8 jpgbuf[2048];

jpg_test = (struct jpg_device*)dev_get(HG_JPG_DEVID);

/*
Obtain the handle of jpg through dev_get(), configure the interrupt registration function of the jpg module, and execute it at the end of the frame.

Break function jpg_done_isr, pass parameter as jpg_test

*/

jpg_request_irq(jpg_test, (jpg_irq_hdl)&jpg_done_isr,BIT(0),jpg_test);

```

7.4.9. jpg_release_irq()

jpg module quantization table update

Function prototype

```
int32 jpg_release_irq(struct jpg_device *p_jpg,uint32 irq_flags)
```

Function parameters

parameter	type	illustrate
p_jpg	struct jpg_device	The handle of the JPG. Usually use dev_get() to get the handle
irq_flags uint32		Jpg module subinterrupt number BIT (0): frame completion interrupt BIT(1): Frame buf data full interrupt BIT(2): Frame error interrupt

Return value

return value	type	illustrate
RET_OK	int32	The jpg module fine-tuned the quantization table successfully
RET_ERR	int32	jpg module failed to fine-tune quantization table

Code examples

```
struct jpg_device *jpg_test = NULL;

extern char quality_tab[6][128];

jpg_test = (struct jpg_device*)dev_get(HG_JPG_DEVID);

/*
Get the jpg handle through dev_get() and release the frame end interrupt
*/

jpg_release_irq(jpg_test,BIT(0));
```

8. CRC interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of CRC. They are located in "sdk\include\hal" file directory

8.1. Enum

8.1.1. CRC_DEV_FLAGS

enum CRC_DEV_FLAGS describes the CRC working mode.

enumerator	illustrate
CRC_DEV_FLAGS_CONTINUE_CALC	CRC works in continuous operation mode and is mainly used for multiple non-continuous data buffers Calculate total CRC @Note: In this mode, the CRC polynomial configuration will inherit the previously configured polynomial.

8.1.2. CRC_DEV_TYPE

enum CRC_DEV_TYPE describes the CRC check mode.

enumerator	illustrate	
CRC_TYPE_CRC5_USB	CRC-5/USB	x^5+x^2+1
CRC_TYPE_CRC7_MMC	CRC-7/MMC	x^7+x^3+1
CRC_TYPE_CRC8_MAXIM	CRC-8/MAXIM	$x^8+x^5+x^4+1$
CRC_TYPE_CRC8	CRC-8	x^8+x^2+x+1
CRC_TYPE_CRC16	CRC-16/MAXIM	$x^{16}+x^{15}+x^2+1$
CRC_TYPE_CRC16_CCITT	CRC-16/CCITT	$x^{16}+x^{12}+x^5+1$
CRC_TYPE_CRC16_MODBUS	CRC-16/MODBUS	$x^{16}+x^{15}+x^2+1$
CRC_TYPE_CRC32_WINRAR	CRC-32	$x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$
CRC_TYPE_TCPIP_CHKSUM	Calculate TCP/IP packet checksum	

8.2. Define

8.2.1. MACRO

8.3. Structure

8.3.1. crc_dev

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
hold	int32 function pointer	This function locks the CRC resource and is usually used when CRC needs to be calculated multiple times in succession. You need to actively release the hold after use.
calc	int32 function pointer	This function is used to calculate CRC

8.4. Function

8.4.1. crc_dev_hold()

Lock the CRC resource, usually used when you need to calculate CRC multiple times in a row, and you need to actively release the hold after use.

Function prototype

```
int32 crc_dev_hold(struct crc_dev *dev, uint16 *cookie, uint8 hold)
```

Function parameters

parameter	type	illustrate
Dev	struct uart_device	CRC handle. Usually use dev_get() to get the handle
cookie	uint16 *	The hold can only be released if the cookies are the same.
Hold	uint8	1:hold 0:unHold

Return value

return value	type	illustrate
RET_OK	int32	success
RET_ERR	int32	fail

8.4.2. crc_dev_calc()

Calculate CRC or CheckSum.

Function prototype

```
int32 crc_dev_calc(struct crc_dev *dev, struct crc_dev_req *req, uint32
*crc_val, uint32 flags)
```

Function parameters

parameter	type	illustrate
Dev	struct uart_device	CRC handle. Usually use dev_get() to get the handle

req	struct crc_dev_req *	Configure CRC check request: buffer address, length, CRC type
crc_val	uint32*	Used to return CRC results
flags	uint32	enum CRC_DEV_FLAGS

Return value

return value	type	illustrate
RET_OK	int32	success
RET_ERR	int32	fail

Code examples

```
uint32 crc = 0xffff;

struct crc_dev_req req;

struct crc_dev *crcdev = (struct crc_dev *)dev_get(HG_CRC_DEVID);

if (crcdev) {

    req.type = CRC_TYPE_CRC16_MODBUS;

    req.data = data;

    req.cookie = *data;

    req.len = len;

    crc_dev_hold(crcdev, &req.cookie, 1);

    crc_dev_calc(crcdev, &req, &crc, 0);
```

```

        crc_dev_calc(crcdev, &req, &crc, CRC_DEV_FLAGS_CONTINUE_CALC);

        crc_dev_calc(crcdev, &req, &crc, CRC_DEV_FLAGS_CONTINUE_CALC);

        crc_dev_hold(crcdev, &req.cookie, 0);

    } else {

        os_printf("no crc dev\n");

    }

```

9. AES interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of AES. They are located in "sdk\include\hal" file directory

9.1. Enum

9.1.1. SYSAES_KEY_LEN

enum SYSAES_KEY_LEN describes the AES key length.

enumerator	illustrate
AES_KEY_LEN_BIT_128	AES key length: 128 bit @Note:
AES_KEY_LEN_BIT_192	AES key length: 192 bit @Note:
AES_KEY_LEN_BIT_256	AES key length: 256bit @Note:

9.1.2. SYSAES_MODE

enum SYSAES_MODE describes the encryption and decryption mode of SYSAES_MODE.

enumerator	illustrate
AES_MODE_ECB	
AES_MODE_CBC	

9.2. Define

9.2.1. MACRO

9.3. Structure

9.3.1. sysaes_para

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
mode	enum SYSAES_MODE	AES encryption and decryption mode
key_len	enum SYSAES_KEY_LEN	AES key length
src	Int8 pointer	Data source address
hand	Int8 pointer	Data destination address
block_num	uint32	Number of AES Blocks (1 Block equals 16 Byte)
Key	Int8 pointer	AES key address
Iv	Int8 pointer	AES IV address (used in AES CBC mode)

9.3.2. sysaes_dev

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
encrypt	int32 function pointer	AES encryption
decrypt	int32 function pointer	AES decryption

9.4. Function

9.4.1. sysaes_encrypt

AES encryption.

Function prototype

```
int32(*encrypt)(struct sysaes_dev *dev, struct sysaes_para *para);
```

Function parameters

parameter	type	illustrate
Dev	struct sysaes_dev *	AES handle. Usually use dev_get() to get the handle
for	struct sysaes_para *	AES encryption data address, key address, encryption mode and other configurations

Return value

return value	type	illustrate
RET_OK	int32	success
RET_ERR	int32	fail

9.4.2. sysaes_decrypt()

AES decryption.

Function prototype

```
int32(*decrypt)(struct sysaes_dev *dev, struct sysaes_para *para);
```

Function parameters

parameter	type	illustrate
Dev	struct sysaes_dev *	AES handle. Usually use dev_get() to get the handle
for	struct sysaes_para * AES decryption data address, key address, encryption mode and other configurations	

Return value

return value	type	illustrate
RET_OK	int32	success
RET_ERR	int32	fail

Code examples

```
struct sysaes_para para;

uint8 *src = os_malloc(ALIGN(len, 16));

uint8 *dest = os_malloc(ALIGN(len, 16));

uint8 *k = os_malloc(ALIGN(len, 16));

struct sysaes_dev *aes = (struct sysaes_dev *)dev_get(HG_HWAES_DEVID);

if (aes && key && src && dest) {

    para.mode = AES_MODE_ECB;
```

```

    to.src      = src;

    para.hand = hand;

    para.block_num = (len + 15) / 16; //round up

    para.key = k;

    ret = (en ? sysaes_encrypt(aes, &para) : sysaes_decrypt(aes,
        &for));

    if (right == RET_OK) {

        os_memcpy(out, dest, len);

    }

}

if(src) os_free(src);

if(dest) os_free(dest);

if(k) os_free(k);

```

10. SDHOST interface description

10.1. Enum

10.2. Define

10.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
-----------	------	------------

Parameter 1		
Parameter 2		

10.3. Structure

10.3.1. sdh_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
heat	int32 function pointer	Turn on the jpg clock and initialize each table
open	int32 function pointer	Turn on sdhost function
close	int32 function pointer	Turn off sdhost function
ioctl	int32 function pointer	sdhost configuration information, function call
cmd	int32 function pointer	sdhost command sends
write	int32 function pointer	DMA sends data to sd slave
read	int32 function pointer	Read data from sd slave
freq_min	uint32	Minimum rate, currently 400K, for initialization
freq_max	uint32	maximum rate
valid_ocr	uint32	voltage range
ioctg	rt_mmc sd_io_cfg structure	Sdhost configuration information
card_capacity	uint32	card capacity
new_lba	uint32	The latest read and write operation address

sd_opt	uint8	Read and write status identifier
data	Data information of rt_mmcscd_data structure read and write operations	
dat_sema	os_semaphore	Used to wake up after read and write operations are completed

10.4. Function

10.4.1. sdhost_io_func_init()

Initialization of io function of sd host

Function prototype

```
void sdhost_io_func_init(uint32 req)
```

Function parameters

parameter	type	illustrate
req	uint32	0:1 line mode IO initialization
		1:4 line mode IO initialization

Code examples

```
struct jpg_device *sdh_test = NULL;

sdh_test = (struct sdh_device*)dev_get(HG_SDC_HOST_DEVID);

/*
Obtain the handle of SDH through dev_get(), determine whether sdh is configured in 4-wire mode, and enable the corresponding IO
*/

sdhost_io_func_init(sdh_test->flags&MMCSDBUSWIDTH_4);
```

10.4.2. sdhost cmd

Sdhost command send operation

Function prototype

```
int32 (*cmd)(struct sdh_device *sdhost,struct rt_mmcsd_cmd *cmd);
```

Function parameters

parameter	type	illustrate
sdhost	struct sdh_device	Handle to SDHOST. Usually use dev_get() to get the handle
cmd	rt_mmcsd_cmd	Command parameter structure, command instruction value, response value, parameter value

Return value

return value	type	illustrate
RET_OK	int32	Sdhost command sent successfully
RET_ERR	int32	Sdhost command failed to send

Code examples

```
/*Send cmd7*/

uint32 send_select_card(struct sdh_device * host)
{
    struct rt_mmcsd_cmd cmd;

    int ret = 0;

    memset(&cmd, 0, sizeof(struct rt_mmcsd_cmd));

    cmd.cmd_code = SELECT_CARD;

    if (host->rca)
    {
        cmd.arg = host->rca << 16;

        cmd.flags = RESP_R1 | CMD_AC;
```

```

    }

    else

    {

        cmd.arg = 0;

        cmd.flags = RESP_NONE | CMD_AC;

    }

    if(host->cmd)

        ret = host->cmd(host,&cmd);

    return right;

}

```

10.4.3. sd_multiple_read()

sd host sends read command to read data

Function prototype

```

int sd_multiple_read(struct sdh_device * host,
                    uint32 lba,
                    uint32 len,
                    uint8* buf
                    )

```

Function parameters

parameter	type	illustrate
host	struct sdh_device	Handle to SDHOST. Usually use dev_get() to get the handle
lba	uint32	Read card address
only	uint32	length of data to read
buf	uint8 *	The location where the data is read. This location cannot be set to psram.

Return value

return value	type	illustrate
RET_OK	int32	Sdhost read data successfully
RET_ERR	int32	Sdhost failed to read data

Code examples

```

struct sd_device *sdh_test = NULL;

uint8 buf[1024];

sdh_test = (struct sdh_device*)dev_get(HG_SDC_HOST_DEVID);

/*
Obtain the handle of SDH through dev_get(), read the data starting from sector No. 4, read 1024, and store
into buf
*/

sd_multiple_read((struct sdh_device*)sdh_test,4,1024,buf);

```

10.4.4. sd_multiple_write()

sd host sends write command to write data

Function prototype

```
int sd_multiple_write(struct sdh_device * host,
                    uint32 lba,
                    uint32 len,
                    uint8* buf
                    )
```

Function parameters

parameter	type	illustrate
host	struct sdh_device	Handle to SDHOST. Usually use dev_get() to get the handle
lba	uint32	Card address written
only	uint32	Length of data to be written
buf	uint8 *	The source data location to write data to. This location cannot be set to psram.

Return value

return value	type	illustrate
RET_OK	int32	Sdhost writes data successfully
RET_ERR	int32	Sdhost failed to write data

Code examples

```
struct sd_device *sdh_test = NULL;

sdh_test = (struct sdh_device*)dev_get(HG_SDC_HOST_DEVID);

/*

Obtain the handle of SDH through dev_get() and write the data starting from sector No. 4. The amount of data written is 1024.

The source data is in source_buf

*/
```

```
sd_multiple_write((struct sdh_device*)sdh_test,4,1024,source_buf);
```

10.4.5. sd_set_clk()

Set the output clock of sd host

Function prototype

void sd_set_clk(struct sdh_device * host,uint32_t clk)

Function parameters

parameter	type	illustrate
host	struct sdh_device	Handle to SDHOST. Usually use dev_get() to get the handle
clk	uint32_t	Configure sd host clock frequency

Code examples

```
struct sd_device *sdh_test = NULL;

sdh_test = (struct sdh_device*)dev_get(HG_SDC_HOST_DEVID);

/*

Obtain the SDH handle through dev_get() and configure the output clock of the sd host to 24M

*/

sd_set_clk((struct sdh_device*)sdh_test,24000000);
```

10.4.6. sd_set_bus_width()

Configure the bus of sd host as 1 line or 4 lines

Function prototype

```
void sd_set_bus_width(struct sdh_device * host, uint32_t width)
```

Function parameters

parameter	type	illustrate
host	struct sdh_device	Handle to SDHOST. Usually use dev_get() to get the handle
width	uint32_t	Line 1: MMCSD_BUS_WIDTH_1 Line 4: MMCSD_BUS_WIDTH_4

Code examples

```
struct sd_device *sdh_test = NULL;

sdh_test = (struct sdh_device*)dev_get(HG_SDC_HOST_DEVID);

/*
Obtain the SDH handle through dev_get() and configure the 1-wire mode of the sd host
*/

sd_set_bus_width((struct sdh_device*)sdh_test, MMCSD_BUS_WIDTH_1);
```

10.4.7. Sd open

Sd host module function is turned on

Function prototype

```
int32 (*open)(struct sdh_device *sdhost,uint8 bus_w);
```

Function parameters

parameter	type	illustrate
sdhost	struct sdh_device	Handle to SDHOST. Usually use dev_get() to get the handle
bus_w	uint8	Bus configuration, 1 is 1 line, 4 is 4 lines

value

return value	type	illustrate
RET_OK	int32	Sdhost opened successfully
RET_ERR	int32	Sdhost failed to open

Code examples

```
struct jpg_device *sdh_test = NULL;

sdh_test = (struct sdh_device*)dev_get(HG_SDC_HOST_DEVID);

/*

Obtain the handle of SDH through dev_get(), open the sdh module and enable it as 1 line

*/

sdh_test->open(sdh_test,1)
```

10.4.8. Sd close

Sd host module function is turned on

Function prototype

```
int32 (*close)(struct sdh_device *sdhost);
```

Function parameters

parameter	type	illustrate
sdhost	struct sdh_device	Handle to SDHOST. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	Sdhost closed successfully
RET_ERR	int32	Sdhost shutdown failed

Code examples

```
struct jpg_device *sdh_test = NULL;

sdh_test = (struct sdh_device*)dev_get(HG_SDC_HOST_DEVID);

/*

Obtain the SDH handle through dev_get(), and the sdh module is closed

*/

sdh_test->close(sdh_test)
```

11. I2C interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of I2C. They are located in the "" file directory

11.1. Enum

11.1.1. i2c_mode

i2c_mode describes the operating mode.

enumerator	illustrate
IIC_MODE_MASTER	I2C works in host mode
IIC_MODE_SLAVE	I2C works in slave mode

11.1.2. i2c_addr_mode

i2c_mode describes the address format. In I2C master mode, it indicates the address format of the slave; in I2C slave mode,

Represents its own address format.

enumerator	illustrate
IIC_ADDR_7BIT	I2C address format is 7bit
IIC_ADDR_10BIT	I2C address format is 10bit

11.1.3. i2c_ioctl_cmd

i2c_ioctl_cmd describes the I2C configuration command, which is configured by calling i2c_ioctl().

enumerator	illustrate
IIC_SDA_OUTPUT_DELAY	I2C's SDA delayed output data, only used in I2C master mode
IIC_STRONG_OUTPUT	I2C turns on or off strong output mode @Note: Cannot connect I2C devices that are both in strong output mode
IIC_FILTERING	I2C turns filtering on or off

IIC_SET_DEVICE_ADDR	Set address. In master mode, set the slave address; in slave mode, Set your own address.
---------------------	---

11.1.4. i2c_irq_flag

i2c_irq_flag describes the interrupt types that I2C can apply for, through i2c_request_irq() and i2c_release_irq() function is used.

enumerator	illustrate
I2C_IRQ_FLAG_TX_DONE	I2C transmission completion interrupt
I2C_IRQ_FLAG_RX_DONE	I2C receive completion interrupt
I2C_IRQ_FLAG_RX_NACK	NACK received interrupt
I2C_IRQ_FLAG_RX_ERROR	Receive BUFFER overflow interrupt, the threshold is 40bit
I2C_IRQ_FLAG_DETECT_STOP	A STOP signal interrupt is detected on the bus

11.2. Define

11.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

11.3. Structure

11.3.1. i2c_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type	illustrate
-------------------	------------

dev	struct dev_obj	Describes operating system related content
open	int32 function pointer	
close	int32 function pointer	
send_stop	int32 function pointer	
baudrate	int32 function pointer	
read	int32 function pointer	
write	int32 function pointer	
request_irq	int32 function pointer	
release_irq	int32 function pointer	

11.4. Function

11.4.1. i2c_open()

Initialize I2C. This function must be called before using the I2C module. After I2C initialization is successful, the default

The default configuration is: disable the strong output function. If you need to change the configuration, you can use i2c_ioctl after the initialization is completed.
function to make changes.

Function prototype

```
int32 i2c_open( struct i2c_device *i2c,
               enum i2c_mode          mode,
               enum i2c_addr_mode addr_mode,
               uint32                  addr
               )
```

Function parameters

parameter	type	illustrate
i2c	struct i2c_device	I2C handle. Usually use dev_get() to get the handle

mode	enum i2c_mode	I2C working mode, refer to enumeration i2c_mode
addr_mode	enum i2c_addr_mode	I2C address format, refer to enumeration i2c_addr_mode
addr	uint32	The address of the I2C device @Note: I2C master mode, represents the address format of the slave; I2C slave mode, represents its own address format

Return value

return value	type	illustrate
RET_OK	int32	I2C module initialization successful
RET_ERR	int32	I2C module initialization failed

Code examples

```

struct i2c_device *i2c_test = NULL;

i2c_test = (struct i2c_device*)dev_get(HG_I2C2_DEVID);

/* Obtain the handle of I2C2 through dev_get(), and configure the host mode, 7-bit address mode, and slave device
The address is 0x80 */

i2c_open(i2c_test, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x80);

```

11.4.2. i2c_close()

Turn off I2C. After calling this function, I2C will not work properly and you need to use the i2c_open() function again to open.

Function prototype

```
int32 i2c_close(struct i2c_device *i2c)
```

Function parameters

parameter	type	illustrate
i2c	struct i2c_device	I2C handle. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	I2C module closed successfully
RET_ERR	int32	I2C module shutdown failed

Code examples

```

struct i2c_device *i2c_test = NULL;

i2c_test = (struct i2c_device*)dev_get(HG_I2C2_DEVID);

/* Obtain the handle of I2C2 through dev_get(), and configure the host mode, 7-bit address mode, and slave device
The address is 0x80 */

i2c_open(i2c_test, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x80);

/* Close I2C */

i2c_close(i2c_test);

```

11.4.3. i2c_set_baudrate()

Set the I2C baud rate. The I2C module must be configured with a baud rate before it can be used normally.

Function prototype

```
int32 i2c_set_baudrate(struct i2c_device *i2c, uint32 baudrate)
```

Function parameters

parameter	type	illustrate
i2c	struct i2c_device	I2C handle. Usually use dev_get() to get the handle
baudrate	uint32	I2C set baud rate value

Return value

return value	type	illustrate
RET_OK	int32	I2C module sets baud rate successfully
RET_ERR	int32	I2C module failed to set baud rate

Code examples

```
struct i2c_device *i2c_test = NULL;

i2c_test = (struct i2c_device*)dev_get(HG_I2C2_DEVID);

/* Obtain the handle of I2C2 through dev_get(), and configure the host mode, 7-bit address mode, and slave device
The address is 0x80 */

i2c_open(i2c_test, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x80);

/*Set the baud rate to 300KHz */

i2c_set_baudrate(i2c_test, 300000);
```

11.4.4. i2c_ioctl()

According to the command in enum adc_ioctl_cmd, call this function to configure the I2C module.

Function prototype

```
int32 i2c_ioctl(struct i2c_device *i2c, uint32 cmd, uint32 param)
```

Function parameters

parameter	type	illustrate
i2c	struct i2c_device	I2C handle. Usually use dev_get() to get the handle
cmd	enum i2c_mode	For configuration commands of the I2C module, see enumeration i2c_ioctl_cmd
param	uint32	Configuration parameters, depending on the configuration command

Return value

return value	type	illustrate
RET_OK	int32	I2C module configured successfully
RET_ERR	int32	I2C module configuration failed

Code examples

```
struct i2c_device *i2c_test = NULL;

i2c_test = (struct i2c_device*)dev_get(HG_I2C2_DEVID);

/* Obtain the handle of I2C2 through dev_get(), and configure the host mode, 7-bit address mode, and slave device
The address is 0x80 */

i2c_open(i2c_test, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x80);

/*Set the baud rate to 300KHz */

i2c_set_baudrate(i2c_test, 300000);

/*Set to strong output mode*/

i2c_ioctl(i2c_test, IIC_STRONG_OUTPUT, 1);
```

11.4.5. i2c_write()

I2C sends data according to the relevant configuration.

Note:

- 1. In host mode, before sending data, the I2C module will first send the slave device address (slave device address Configured by i2c_open() function).

Function prototype

```
int32 i2c_write(struct i2c_device *i2c,
               you8 *addr,
               uint32 addr_len,
               you8 *buf,
               uint32 buf_len
               )
```

Function parameters

parameter	type	illustrate
i2c	struct i2c_device	I2C handle. Usually use dev_get() to get the handle
addr	you8*	The address to be accessed, only valid for host mode
addr_len	uint32	Address length, unit is byte, only valid for host mode
buf	you8*	The starting address of the data BUFFER to be sent

buf_len	uint32	Data length, unit is byte
---------	--------	---------------------------

Return value

return value	type	illustrate
RET_OK	int32	I2C module sent successfully
RET_ERR	int32	I2C module failed to send

Code examples

```
uint8 tx_buf[256];

uint32 i = 0;

uint8 addr = 0x30;

/* Initialize tx_buf, the value has no practical meaning*/

for (i = 0; i < sizeof(tx_buf)/sizeof(tx_buf[0]); i++) {

    tx_buf[i] = 0x55;

}

struct i2c_device *i2c_test = NULL;

i2c_test = (struct i2c_device*)dev_get(HG_I2C2_DEVID);

/* Obtain the handle of I2C2 through dev_get(), and configure the host mode, 7-bit address mode, and slave device

The address is 0x80 */

i2c_open(i2c_test, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x80);

/*Set the baud rate to 300KHz */

i2c_set_baudrate(i2c_test, 300000);

/* Write 256byte data to the I2C slave device with device address 0x80, starting from address 0x30*/

i2c_write(i2c_test, &addr, 1, tx_buf, sizeof(tx_buf)/sizeof(tx_buf[0]));
```

11.4.6. i2c_read()

I2C receives based on the relevant configuration of the received data.

Note:

- 1. In host mode, before receiving data, the I2C module will first send the slave device address (slave device address

Configured by i2c_open() function).

Function prototype

```
int32 i2c_read( struct i2c_device *i2c,
               you8 *addr,
               uint32 addr_len,
               you8 *buf,
               uint32 buf_len
               )
```

Function parameters

parameter	type	illustrate
i2c	struct i2c_device	I2C handle. Usually use dev_get() to get the handle
addr	you8*	The address to be accessed, only valid for host mode
addr_len	uint32	Address length, unit is byte, only valid for host mode
buf	you8*	The starting address of the data BUFFER to be received
buf_len	uint32	Data length, unit is byte

Return value

return value	type	illustrate
RET_OK	int32	I2C module received successfully
RET_ERR	int32	I2C module reception failed

Code examples

```
uint8 rx_buf[256];

uint32 i = 0;

uint8 addr = 0x30;

/* Initialize rx_buf = 0 */

for (i = 0; i < sizeof(rx_buf)/sizeof(rx_buf[0]); i++) {
    rx_buf[i] = 0x00;
}
```

```
}

struct i2c_device *i2c_test = NULL;

i2c_test = (struct i2c_device*)dev_get(HG_I2C2_DEVID);

/* Obtain the handle of I2C2 through dev_get(), and configure the host mode, 7-bit address mode, and slave device
The address is 0x80 */

i2c_open(i2c_test, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x80);

/*Set the baud rate to 300KHz */

i2c_set_baudrate(i2c_test, 300000);

/* Read 256byte data starting from address 0x30 of the I2C slave device with device address 0x80*/

i2c_read(i2c_test, &addr, 1, rx_buf, sizeof(rx_buf)/sizeof(rx_buf[0]));
```

11.4.7. i2c_request_irq()

According to the interrupt type of enum i2c_irq_flag, call this function to apply for the interrupt of the I2C module.

Function prototype

```
int32 i2c_request_irq(struct i2c_device *i2c,
                    i2c_irq_hdl handle,
                    uint32 irq_data,
                    uint32 irq_flag,
                    )
```

Function parameters

parameter	type	illustrate
i2c	struct i2c_device	I2C handle. Usually use dev_get() to get the handle
handle	i2c_irq_hdl	Interrupt handle, executed after the interrupt is generated
irq_data	uint32	Interrupt handle parameters
irq_flag	uint32	The requested interrupt type, refer to the enumeration i2c_irq_flag

Return value

return value	type	illustrate
--------------	------	------------

RET_OK	int32	I2C module applied for interrupt successfully
RET_ERR	int32	I2C module failed to apply for interrupt

Code examples

```
void i2c_irq_hdl(uint32 irq, uint32 irq_data, uint32 param) {  
    __NOP();  
}  
  
struct i2c_device *i2c_test = NULL;  
  
i2c_test = (struct i2c_device*)dev_get(HG_I2C2_DEVID);  
  
/* Obtain the handle of I2C2 through dev_get(), and configure the host mode, 7-bit address mode, and slave device  
The address is 0x80 */  
  
i2c_open(i2c_test, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x80);  
  
/*Set the baud rate to 300KHz */  
  
i2c_set_baudrate(i2c_test, 300000);  
  
/* Apply for I2C transmission completion interrupt*/  
  
i2c_request_irq(i2c_test, i2c_irq_hdl, 0, I2C_IRQ_FLAG_TX_DONE);
```

11.4.8. i2c_release_irq()

According to the interrupt type of enum i2c_irq_flag, call this function to release the interrupt of the I2C module.

Function prototype

```
i2c_release_irq(struct i2c_device *i2c, uint32 irq_flag)
```

Function parameters

parameter	type	illustrate
i2c	struct i2c_device	I2C handle. Usually use dev_get() to get the handle
irq_flag	uint32	Released interrupt type, refer to enumeration i2c_irq_flag

Return value

return value	type	illustrate
RET_OK	int32	I2C module interrupt closed successfully
RET_ERR	int32	I2C module interrupt shutdown failed

Code examples

```
struct i2c_device *i2c_test = NULL;

i2c_test = (struct i2c_device*)dev_get(HG_I2C2_DEVID);

/* Obtain the handle of I2C2 through dev_get(), and configure the host mode, 7-bit address mode, and slave device
The address is 0x80 */

i2c_open(i2c_test, IIC_MODE_MASTER, IIC_ADDR_7BIT, 0x80);

/*Set the baud rate to 300KHz */

i2c_set_baudrate(i2c_test, 300000);

/* Release I2C transmission completion interrupt */

i2c_release_irq(i2c_test, I2C_IRQ_FLAG_TX_DONE);
```

12. SPI interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of SPI. They are located in the "" file directory

12.1. Enum

12.1.1. spi_work_mode

spi_mode describes the operating mode.

enumerator	illustrate
SPI_MASTER_MODE	SPI works in host mode
SPI_SLAVE_MODE	SPI works in slave mode
SPI_SLAVE_FSM_MODE	SPI works in slave state machine mode

12.1.2. spi_wire_mode

spi_mode describes the line mode of operation.

enumerator	illustrate
SPI_WIRE_SINGLE_MODE	SPI works in 1 DATA line mode @Note: At this time, SPI includes: CLK; CS; DATA lines. One CLK cycle transmits 1 bit data
SPI_WIRE_NORMAL_MODE	SPI works in 2 DATA line mode, which is standard SPI mode @Note: At this time, SPI includes: CLK; CS; DATA0(MOSI); DATA1(MISO) lines. One CLK cycle transmits Lose 1 bit
SPI_WIRE_DUAL_MODE	SPI works in 2 DATA line mode @Note: At this time, SPI includes: CLK; CS; DATA0; DATA1 lines. One CLK cycle transmits 2 bits
SPI_WIRE_QUAD_MODE	SPI works in 4 DATA line mode @Note: At this time, SPI includes: CLK; CS; DATA0; DATA1; DATA2; DATA3 lines. One CLK cycle Transmit 4bit

12.1.3. spi_clk_mode

spi_clk_mode describes the clock mode of operation.

enumerator	illustrate
SPI_CPOL_0_CPHA_0	SPI works on the rising edge for sampling and falling edge for transmission. When idle, CLK is low level. @Note: SPI_CLK_MODE_0 = SPI_CPOL_0_CPHA_0
SPI_CPOL_0_CPHA_1	SPI works on the falling edge for sampling and rising edge for transmission. When idle, CLK is low level. @Note: SPI_CLK_MODE_1 = SPI_CPOL_0_CPHA_1
SPI_CPOL_1_CPHA_0	SPI works on the falling edge for sampling and rising edge for transmission. When idle, CLK is high level. @Note: SPI_CLK_MODE_2 = SPI_CPOL_1_CPHA_0
SPI_CPOL_1_CPHA_1	SPI works on the rising edge for sampling and falling edge for transmission. When idle, CLK is high level. @Note: SPI_CLK_MODE_3 = SPI_CPOL_1_CPHA_1

12.1.4. spi_ioctl_cmd

spi_ioctl_cmd describes the SPI configuration command, which is configured by calling spi_ioctl().

enumerator	illustrate
SPI_WIRE_MODE_SET	SPI sets the line mode of operation
SPI_WIRE_MODE_GET	Get the current line mode of SPI operation
SPI_SAMPLE_DELAY	SPI sampling delay, only valid for SPI host
SPI_SET_FRAME_SIZE	Set the data length of an SPI frame in bits
SPI_SET_LEN_THRESHOLD	Set the data length threshold sent by SPI using DMA, in byte @Note: The default threshold is 5byte

12.1.5. spi_irq_flag

spi_irq_flag describes the interrupt types that SPI can apply for, through spi_request_irq() and spi_release_irq() function is used.

enumerator	illustrate
SPI_IRQ_FLAG_TX_DONE	SPI send data completion interrupt
SPI_IRQ_FLAG_RX_DONE	SPI receive data completion interrupt
SPI_IRQ_FLAG_FIFO_OVERFLOW	SPI hardware FIFO overflow interrupt, FIFO capacity is 40 bits
SPI_IRQ_FLAG_CS_RISING	Detects rising edge interrupt of CS, only valid for SPI slave
SPI_IRQ_FLAG_SLAVE_FSM_READ_STATUS	SPI receives read status command interrupt, only valid for SPI slave state machine

12.2. Define

12.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

12.3. Structure

12.3.1. spi_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
open	int32 function pointer	
close	int32 function pointer	
ioctl	int32 function pointer	

read	int32 function pointer	
write	int32 function pointer	
request_irq	int32 function pointer	
release_irq	int32 function pointer	

12.4. Function

12.4.1. spi_open()

Initialize SPI. This function must be called before using the SPI module. After SPI initialization is successful, the default configuration is considered to be: data high bit sent first; CS output low level is active; the length of one frame is 8 bits. If you need to change the configuration, you can After initialization is complete, use the spi_ioctl function to make changes.

Function prototype

```
int32 spi_open( struct spi_device *p_spi,
               uint32      clk_freq,
               uint32      work_mode,
               uint32      wire_mode,
               uint32      clk_mode
               )
```

Function parameters

parameter	type	illustrate
p_spi	struct spi_device	SPI handle. Usually use dev_get() to get the handle
clk_freq	uint32	SPI operating clock frequency configuration
work_mode	uint32	SPI working mode, refer to the enumeration spi_work_mode
wire_mode	uint32	SPI working wire mode, refer to the enumeration spi_wire_mode
clk_mode	uint32	SPI working clock mode, refer to the enumeration spi_clk_mode

Return value

return value	type	illustrate
RET_OK	int32	SPI module initialization successful
RET_ERR	int32	SPI module initialization failed

Code examples

```
struct spi_device *spi_test = NULL;

spi_test = (struct spi_device*)dev_get(HG_SPI0_DEVID);

/* Obtain the handle of SPI0 through dev_get() and configure the host mode, CLK=10MHz,
Line mode is standard SPI mode, clock mode is MODE0

*/

spi_open(spi_test, 10000000, SPI_MASTER_MODE, \
SPI_WIRE_NORMAL_MODE, SPI_CPOL_0_CPHA_0);
```

12.4.2. spi_close()

Turn off SPI. After calling this function, SPI will not work properly and you need to use the spi_open() function to open again.

Function prototype

```
int32 spi_close(struct spi_device *p_spi)
```

Function parameters

parameter	type	illustrate
p_spi	struct spi_device	SPI handle. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	SPI module closed successfully
RET_ERR	int32	SPI module shutdown failed

Code examples

```
struct spi_device *spi_test = NULL;

spi_test = (struct spi_device*)dev_get(HG_SPI0_DEVID);

/* Obtain the handle of SPI0 through dev_get() and configure the host mode, CLK=10MHz,
Line mode is standard SPI mode, clock mode is MODE 0

*/

spi_open(spi_test, 10000000, SPI_MASTER_MODE, \
SPI_WIRE_NORMAL_MODE, SPI_CPOL_0_CPHA_0);

/* Close SPI */

spi_close(spi_test);
```

12.4.3. spi_ioctl()

According to the command in enum spi_ioctl_cmd, call this function to configure the SPI module.

Function prototype

```
int32 spi_ioctl(struct spi_device *p_spi,
                uint32 cmd,
                uint32 param1,
                uint32 param2
                )
```

Function parameters

parameter	type	illustrate
p_spi	struct spi_device	SPI handle. Usually use dev_get() to get the handle
cmd	uint32	Configuration commands for SPI module
param1	uint32	Configuration parameter 1, determined according to the configuration command
param2	uint32	Configuration parameter 2, determined according to the configuration command

Return value

return value	type	illustrate
--------------	------	------------

RET_OK	int32	SPI module configured successfully
RET_ERR	int32	SPI module configuration failed

Code examples

```
uint8 rx_buf[100];

memset((void *)rx_buf, 0, sizeof(rx_buf)/sizeof(rx_buf[0]));

struct spi_device *spi_test = NULL;

spi_test = (struct spi_device*)dev_get(HG_SPI0_DEVID);

/* Obtain the handle of SPI0 through dev_get() and configure the host mode, CLK=10MHz,
Line mode is standard SPI mode, clock mode is MODE 0

*/

spi_open(spi_test, 10000000, SPI_MASTER_MODE, \
SPI_WIRE_NORMAL_MODE, SPI_CPOL_0_CPHA_0);

/* Set frame size to 16bit */

spi_ioctl(spi_test, SPI_SET_FRAME_SIZE, 16, 0);
```

12.4.4. spi_read()

SPI receives data based on the relevant configuration.

Note:

- 1. In master mode, after calling this function, CLK is generated immediately to receive data from the slave.
- 2. In slave mode, after calling this function, the slave waits for the CLK of the master before receiving it.

Function prototype

```
int32 spi_read(struct spi_device *p_spi, void *buf, uint32 size)
```

Function parameters

parameter	type	illustrate
p_spi	struct spi_device	SPI handle. Usually use dev_get() to get the handle
buf	void*	The starting address of the data BUFFER to be received
size	uint32	Data length, unit is byte

		@Note: The maximum data length is: 4095
--	--	--

Return value

return value	type	illustrate
RET_OK	int32	SPI module received successfully
RET_ERR	int32	SPI module reception failed

Code examples

```
uint8 rx_buf[100];

memset((void *)rx_buf, 0, sizeof(rx_buf)/sizeof(rx_buf[0]));

struct spi_device *spi_test = NULL;

spi_test = (struct spi_device*)dev_get(HG_SPI0_DEVID);

/* Obtain the handle of SPI0 through dev_get() and configure the host mode, CLK=10MHz,
Line mode is standard SPI mode, clock mode is MODE 0
*/

spi_open(spi_test, 10000000, SPI_MASTER_MODE, \
SPI_WIRE_NORMAL_MODE, SPI_CPOL_0_CPHA_0);

/* SPI host, receive 100byte */

spi_read(spi_test, (void *)rx_buf, 100);
```

12.4.5. spi_write()

SPI sends data according to the relevant configuration.

Note:

- 1. In host mode, after calling this function, CLK is generated immediately to send data.
- 2. In slave mode, after calling this function, the slave waits for the CLK of the master before sending.

Function prototype

```
int32 spi_write(struct spi_device *p_spi, const void *buf, uint32 size)
```

Function parameters

parameter	type	illustrate
-----------	------	------------

p_spi	struct spi_device	SPI handle. Usually use dev_get() to get the handle
buf	const void*	The starting address of the data BUFFER to be sent
size	uint32	Data length, unit is byte @Note: The maximum data length is: 4095

Return value

return value	type	illustrate
RET_OK	int32	SPI module sent successfully
RET_ERR	int32	SPI module failed to send

Code examples

```
uint8 tx_buf[100];

/* Initialize tx_buf, no practical significance*/

memset((void *)tx_buf, 0x55, sizeof(tx_buf)/sizeof(tx_buf[0]));

struct spi_device *spi_test = NULL;

spi_test = (struct spi_device*)dev_get(HG_SPI0_DEVID);

/* Obtain the handle of SPI0 through dev_get() and configure the host mode, CLK=10MHz,
Line mode is standard SPI mode, clock mode is MODE 0

*/

spi_open(spi_test, 10000000, SPI_MASTER_MODE, \
SPI_WIRE_NORMAL_MODE, SPI_CPOL_0_CPHA_0);

/* SPI host, send 100byte */

spi_write(spi_test, (void *)tx_buf, 100);
```

12.4.6. spi_set_cs()

SPI sets the level value of CS, only in master mode, chip selects SPI slave.

Function prototype

int32 spi_set_cs(struct spi_device *p_spi, uint32 cs, uint32 value)

Function parameters

parameter	type	illustrate
p_spi	struct spi_device	SPI handle. Usually use dev_get() to get the handle
cs	uint32	Reserved, you can fill in any value @Note: Can be used for SPI bus multi-slave expansion: specify a certain SPI slave to be selected
value	uint32	CS level value to be set

Return value

return value	type	illustrate
RET_OK	int32	The SPI module sets the CS level successfully
RET_ERR	int32	SPI module failed to set CS level

Code examples

```
uint8 tx_buf[100];

/* Initialize tx_buf, no practical significance*/

memset((void *)tx_buf, 0x55, sizeof(tx_buf)/sizeof(tx_buf[0]));

struct spi_device *spi_test = NULL;

spi_test = (struct spi_device*)dev_get(HG_SPI0_DEVID);

/* Obtain the handle of SPI0 through dev_get() and configure the host mode, CLK=10MHz,

Line mode is standard SPI mode, clock mode is MODE 0

*/

spi_open(spi_test, 10000000, SPI_MASTER_MODE, \

SPI_WIRE_NORMAL_MODE, SPI_CPOL_0_CPHA_0);

/* SPI host, set CS level to 0 */

spi_set_cs(spi_test, 0, 0);

/* SPI host, send 100byte */
```

```
spi_write(spi_test, (void *)tx_buf, 100);

/* SPI host, set CS level to 1 */

spi_set_cs(spi_test, 0, 1);
```

12.4.7. spi_request_irq()

According to the interrupt type of enum spi_irq_flag, call this function to apply for the interrupt of the SPI module.

Function prototype

```
int32 spi_request_irq(struct spi_device *p_spi,
                    uint32 irq_flag,
                    spi_irq_hdl irqhdl,
                    uint32 irq_data
                    )
```

Function parameters

parameter	type	illustrate
p_spi	struct spi_device	SPI handle. Usually use dev_get() to get the handle
irq_flag	uint32	The requested interrupt type, refer to the enumeration spi_irq_flag
irqhdl	spi_irq_hdl	Interrupt handle, executed after the interrupt is generated
irq_data	uint32	Interrupt handle parameters

Return value

return value	type	illustrate
RET_OK	int32	SPI module applied for interrupt successfully
RET_ERR	int32	SPI module failed to apply for interrupt

Code examples

```
void spi_irq_hdl(uint32 irq, uint32 irq_data){

    __NOP();

}

struct spi_device *spi_test = NULL;

spi_test = (struct spi_device*)dev_get(HG_SPI0_DEVID);

/* Obtain the handle of SPI0 through dev_get() and configure the host mode, CLK=10MHz,

Line mode is standard SPI mode, clock mode is MODE 0

*/

spi_open(spi_test, 10000000, SPI_MASTER_MODE, \

SPI_WIRE_NORMAL_MODE, SPI_CPOL_0_CPHA_0);

/* Apply for SPI transmission completion interrupt*/

spi_request_irq(spi_test, SPI_IRQ_FLAG_TX_DONE, \

spi_irq_hdl, 0);
```

12.4.8. spi_release_irq()

According to the interrupt type of enum spi_irq_flag, call this function to release the interrupt of the SPI module.

Function prototype

```
int32 spi_release_irq(struct spi_device *spi, uint32 irq_flag);
```

Function parameters

parameter	type	illustrate
spi	struct spi_device	SPI handle. Usually use dev_get() to get the handle
irq_flag	uint32	Released interrupt type, refer to the enumeration spi_irq_flag

Return value

return value	type	illustrate
RET_OK	int32	SPI module release interrupt successful
RET_ERR	int32	SPI module release interrupt failed

Code examples

```
void spi_irq_hdl(uint32 irq, uint32 irq_data){
    __NOP();
}

struct spi_device *spi_test = NULL;

spi_test = (struct spi_device*)dev_get(HG_SPI0_DEVID);

/* Obtain the handle of SPI0 through dev_get() and configure the host mode, CLK=10MHz,
Line mode is standard SPI mode, clock mode is MODE 0

*/

spi_open(spi_test, 10000000, SPI_MASTER_MODE, \
SPI_WIRE_NORMAL_MODE, SPI_CPOL_0_CPHA_0);

/* Release SPI transmission completion interrupt*/

spi_request_irq(spi_test, SPI_IRQ_FLAG_TX_DONE);
```

13. TIMER interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of TIMER. They are located in the "" file directory

13.1. Enum

13.1.1. timer_type

timer_type describes the working mode.

enumerator	illustrate
TIMER_TYPE_ONCE	TIMER works in single counting mode @Note: TIMER will stop counting after completing one cycle.
TIMER_TYPE_PERIODIC	TIMER works in loop counting mode @Note: After TIMER counts for one cycle, it will count again from 0

13.1.2. timer_irq_flag

timer_irq_flag describes the interrupt types that TIMER can apply for.

Note:

1. The driver of TIMER automatically turns on the interrupt when the count reaches the period value.
2. Users use timer_request_irq() and timer_release_irq() to apply for and release interrupts without effect.
3. Users use cb and cb_data in timer_start() as interrupt callback functions.

enumerator	illustrate
TIMER_INTR_PERIOD	TIMER counts to period value interrupt

13.1.3. timer_ioctl_cmd

timer_ioctl_cmd is not supported in this chip, so it is not described.

13.2. Define

13.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

13.3. Structure

13.3.1. timer_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
open	int32 function pointer	
close	int32 function pointer	
start	int32 function pointer	
stop	int32 function pointer	
ioctl	int32 function pointer	
request_irq	int32 function pointer	
release_irq	int32 function pointer	

13.4. Function

13.4.1. timer_device_open()

Initialize TIMER. This function must be called before using the TIMER module.

Function prototype

```
int32 timer_device_open(struct timer_device *timer,
                        enum timer_type mode,
                        uint32 flags)
{
}
```

Function parameters

parameter	type	illustrate
timer	struct timer_device	A handle to TIMER. Usually use dev_get() to get the handle
mode	enum timer_type	TIMER working mode selection
flags	uint32	Reserved, not used, can be filled with any value.

Return value

return value	type	illustrate
RET_OK	int32	TIMER module initialized successfully
RET_ERR	int32	TIMER module initialization failed

Code examples

```
struct timer_device *timer_test = NULL;

timer_test = (struct timer_device*)dev_get(HG_TIMER0_DEVID);

/* Get the handle of TIMER0 through dev_get() and configure the single counting mode */

timer_device_open(timer_test, TIMER_TYPE_ONCE, 0);
```

13.4.2. timer_device_close()

Close TIMER. After calling this function, TIMER will not work properly and timer_device_open() needs to be used again

The function opens.

Function prototype

```
int32 timer_device_close(struct timer_device *timer)
```

Function parameters

parameter	type	illustrate
timer	struct timer_device	A handle to TIMER. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	TIMER module closed successfully
RET_ERR	int32	TIMER module failed to close

Code examples

```
struct timer_device *timer_test = NULL;

timer_test = (struct timer_device*)dev_get(HG_TIMER0_DEVID);

/* Get the handle of TIMER0 through dev_get() and configure the single counting mode
*/

timer_device_open(timer_test, TIMER_TYPE_ONCE, 0);

/*Close TIMER module*/

timer_device_close(timer_test);
```

13.4.3. timer_device_start()

Start TIMER timing.

Function prototype

```
int32 timer_device_start( struct timer_device *timer,
```

```
uint32 tmo_us,
timer_cb_hdl cb,
uint32 cb_data
)
```

Function parameters

parameter	type	illustrate
timer	struct timer_device	A handle to TIMER. Usually use dev_get() to get the handle
tmo_us	uint32	Timing time set by TIMER, unit is us
cb	timer_cb_hdl	TIMER count to period value interrupt callback function
cb_data	uint32	callback function parameters

Return value

return value	type	illustrate
RET_OK	int32	TIMER module starts timing successfully
RET_ERR	int32	TIMER module failed to start timing

Code examples

```
void timer_cb_hdl(uint32 cb_data, uint32 irq_flag) {
    __NOP();
}

struct timer_device *timer_test = NULL;

timer_test = (struct timer_device*)dev_get(HG_TIMER0_DEVID);

/* Get the handle of TIMER0 through dev_get() and configure the single counting mode
*/

timer_device_open(timer_test, TIMER_TYPE_ONCE, 0);

/* Start TIMER for 1000us and configure the callback function */

timer_device_start(timer_test, 1000, timer_cb_hdl, 0);
```

13.4.4. timer_device_stop()

Turn off TIMER.

Function prototype

```
int32 timer_device_stop(struct timer_device *timer)
```

Function parameters

parameter	type	illustrate
timer	struct timer_device	A handle to TIMER. Usually use dev_get() to get the handle

Return value

return value	type	illustrate
RET_OK	int32	TIMER module closed timing successfully
RET_ERR	int32	TIMER module failed to close timing

Code examples

```
void timer_cb_hdl(uint32 cb_data, uint32 irq_flag) {  
    __NOP();  
}  
  
struct timer_device *timer_test = NULL;  
  
timer_test = (struct timer_device*)dev_get(HG_TIMER0_DEVID);  
  
/* Get the handle of TIMER0 through dev_get() and configure the single counting mode */  
  
timer_device_open(timer_test, TIMER_TYPE_ONCE, 0);  
  
/* Start TIMER for 1000us and configure the callback function */  
  
timer_device_start(timer_test, 1000, timer_cb_hdl, 0);  
  
/* Close TIMER timing */  
  
timer_device_stop(timer_test);
```

13.4.5. timer_device_ioctl()

The driver has not yet defined an available cmd, and the call is invalid.

Function prototype

```
int32 timer_device_ioctl( struct timer_device *timer,
                        uint32 cmd,
                        uint32 param1,
                        uint32 param2
                        )
```

Function parameters

parameter	type	illustrate
timer	struct timer_device	A handle to TIMER. Usually use dev_get() to get the handle
cmd	uint32	Configuration commands for TIMER module
param1	uint32	Configuration parameter 1, determined according to the configuration command
param2	uint32	Configuration parameter 2, determined according to the configuration command

Return value

return value	type	illustrate
RET_OK	int32	TIMER module configuration successful
RET_ERR	int32	TIMER module configuration failed

Code examples

13.4.6. timer_device_request_irq()

Interrupt related content is configured through timer_device_start(), and this interface call is invalid.

Function prototype

```
int32 timer_request_irq(struct timer_device *timer,
                      uint32 irq_flag,
                      timer_cb_hdl cb,
                      uint32 cb_data
                      )
```

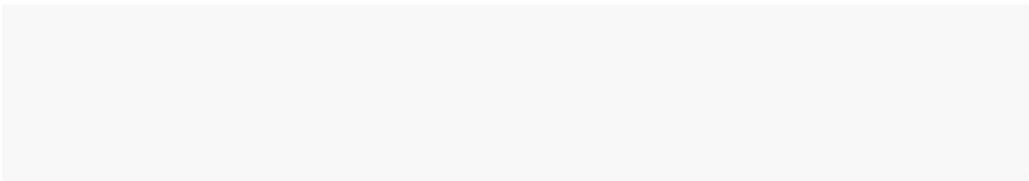
Function parameters

parameter	type	illustrate
timer	struct timer_device	A handle to TIMER. Usually use dev_get() to get the handle
irq_flag	uint32	The requested interrupt type, refer to the enumeration timer_irq_flag
cb	timer_cb_hdl	Interrupt handle, executed after the interrupt is generated
cb_data	uint32	Interrupt handle parameters

Return value

return value	type	illustrate
RET_OK	int32	TIMER module applied for interruption successfully
RET_ERR	int32	TIMER module failed to apply for interrupt

Code examples



13.4.7. timer_device_release_irq()

Interrupt related content is configured through timer_device_start(), and this interface call is invalid.

Function prototype

```
int32 timer_release_irq(struct timer_device *timer, uint32 irq_flag)
```

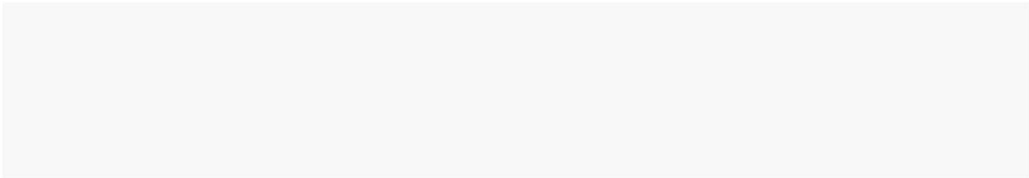
Function parameters

parameter	type	illustrate
timer	struct timer_device	A handle to TIMER. Usually use dev_get() to get the handle
irq_flag	uint32	Released interrupt type, refer to enumeration timer_irq_flag

Return value

return value	type	illustrate
RET_OK	int32	TIMER module release interrupt successful
RET_ERR	int32	TIMER module release interrupt failed

Code examples



14. PWM interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of PWM. They are located in the "" file directory

14.1. Enum

14.1.1. pwm_channel

pwm_channel describes the PWM generation channel.

enumerator	illustrate
PWM_CHANNEL_0 ~ 10	PWM generation channel @Note: Users must bind the channel to the corresponding TIMER in the device.c file.
TIMER_TYPE_PERIODIC	TIMER works in loop counting mode @Note: After TIMER counts for one cycle, it will count again from 0

14.1.2. pwm_irq_flag

pwm_irq_flag describes the interrupt types that PWM can apply for.

enumerator	illustrate
PWM_IRQ_FLAG_COMPARE	PWM channel bound TIMER counts to comparison value interrupt
PWM_IRQ_FLAG_PERIOD	PWM channel bound TIMER counts to period value interrupt

14.1.3. pwm_ioctl_cmd

pwm_ioctl_cmd describes the PWM configuration command, which is configured by calling pwm_ioctl().

enumerator	illustrate
PWM_IOCTL_CMD_SET_PERIOD_DUTY	PWM channel bound TIMER counts to comparison value interrupt
PWM_IRQ_FLAG_PERIOD	PWM channel bound TIMER counts to period value interrupt
PWM_IOCTL_CMD_SET_SINGLE_INCREAM	Set to monotonically increasing mode, only used by binding special TIMER

PWM_IOCTL_CMD_SET_INCREAM_DECREASE is set to increase first and then decrease mode. It can only be used by binding special TIMER.	
---	--

14.2. Define

14.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

14.3. Structure

14.3.1. pwm_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
heat	int32 function pointer	
deinit	int32 function pointer	
start	int32 function pointer	
stop	int32 function pointer	
ioctl	int32 function pointer	
request_irq	int32 function pointer	
release_irq	int32 function pointer	

14.4. Function

14.4.1. pwm_init()

Initialize PWM channel. This function must be called before using the PWM module channel. PWM channel initialization successful

After that, the default configuration is: the TIMER bound to the PWM channel uses the system clock for counting by default.

Function prototype

```
int32 pwm_init( struct pwm_device *pwm,
                enum pwm_channel channel,
                uint32 period_us,
                uint32 h_duty_us
                )
```

Function parameters

parameter	type	illustrate
mod	struct pwm_device	Handle to PWM. Usually use dev_get() to get the handle
channel	enum pwm_channel	PWM channel, refer to enumeration pwm_channel
period_us	uint32	Period value of PWM, unit is us @Note: The reciprocal of the period value is the PWM waveform frequency
h_duty_us	uint32	The time occupied by the high level in the PWM waveform @Note: Its ratio to the period value, that is, the high-level duty cycle

Return value

return value	type	illustrate
RET_OK	int32	PWM channel initialization successful
RET_ERR	int32	PWM channel initialization failed

Code examples

```
struct pwm_device *pwm_test = NULL;

pwm_test = (struct pwm_device*)dev_get(HG_PWM_DEVID);

/* Get the PWM handle through dev_get(), use channel 0,

   Configure period value 100us (10KHz frequency) and high level time (90us)

*/

pwm_init(pwm_test, PWM_CHANNEL_0, 100, 90);
```

14.4.2. pwm_deinit()

Turn off the PWM channel. After calling this function, the PWM channel will not work properly and pwm_init() needs to be used again.
function to initialize.

Function prototype

```
int32 pwm_deinit(struct pwm_device *pwm, enum pwm_channel channel)
```

Function parameters

parameter	type	illustrate
mold	struct pwm_device	Handle to PWM. Usually use dev_get() to get the handle
channel	enum pwm_channel	PWM channel, refer to enumeration pwm_channel

Return value

return value	type	illustrate
RET_OK	int32	PWM channel closed successfully
RET_ERR	int32	PWM channel close failed

Code examples

```
struct pwm_device *pwm_test = NULL;

pwm_test = (struct pwm_device*)dev_get(HG_PWM_DEVID);

/* Get the PWM handle through dev_get(), use channel 0,

   Configure period value 100us (10KHz frequency) and high level time (90us)

*/

pwm_init(pwm_test, PWM_CHANNEL_0, 100, 90);

/* Close PWM channel 0 */

pwm_deinit(pwm_test, PWM_CHANNEL_0);
```

14.4.3. pwm_start()

Start the PWM channel to output PWM waveform. After starting the output, the PWM I/O pin outputs high level by default and continues to The high level time configured by pwm_init().

Function prototype

```
int32 pwm_start(struct pwm_device *pwm, enum pwm_channel channel)
```

Function parameters

parameter	type	illustrate
mold	struct pwm_device	Handle to PWM. Usually use dev_get() to get the handle
channel	enum pwm_channel	PWM channel, refer to enumeration pwm_channel

Return value

return value	type	illustrate
RET_OK	int32	PWM channel started successfully
RET_ERR	int32	PWM channel startup failed

Code examples

```
struct pwm_device *pwm_test = NULL;

pwm_test = (struct pwm_device*)dev_get(HG_PWM_DEVID);

/* Get the PWM handle through dev_get(), use channel 0,

   Configure period value 100us (10KHz frequency) and high level time (90us)

*/

pwm_init(pwm_test, PWM_CHANNEL_0, 100, 90);

/* Start PWM channel 0 to output PWM waveform*/

pwm_start(pwm_test, PWM_CHANNEL_0);
```

14.4.4. pwm_stop()

Stop the PWM channel from outputting PWM waveform.

Function prototype

```
int32 pwm_stop(struct pwm_device *pwm, enum pwm_channel channel)
```

Function parameters

parameter	type	illustrate
mold	struct pwm_device	Handle to PWM. Usually use dev_get() to get the handle
channel	enum pwm_channel	PWM channel, refer to enumeration pwm_channel

Return value

return value	type	illustrate
RET_OK	int32	PWM channel stopped successfully
RET_ERR	int32	PWM channel stop failed

Code examples

```
struct pwm_device *pwm_test = NULL;

pwm_test = (struct pwm_device*)dev_get(HG_PWM_DEVID);

/* Get the PWM handle through dev_get(), use channel 0,

   Configure period value 100us (10KHz frequency) and high level time (90us)

*/

pwm_init(pwm_test, PWM_CHANNEL_0, 100, 90);

/* Start PWM channel 0 to output PWM waveform*/

pwm_start(pwm_test, PWM_CHANNEL_0);

/* Stop PWM channel 0 and output PWM waveform*/

pwm_stop(pwm_test, PWM_CHANNEL_0);
```

14.4.5. pwm_ioctl()

According to the command in enum pwm_ioctl_cmd, call this function to configure the PWM channel.

Function prototype

```
int32 pwm_ioctl(struct pwm_device *pwm,

                enum pwm_channel channel,

                enum pwm_ioctl_cmd ioctl_cmd,

                uint32 param1,

                uint32 param2

                )
```

Function parameters

parameter	type	illustrate
mold	struct pwm_device	Handle to PWM. Usually use dev_get() to get the handle
channel	enum pwm_channel	PWM channel, refer to enumeration pwm_channel
ioctl_cmd	enum pwm_ioctl_cmd	For configuration commands of PWM channels, refer to enumeration pwm_ioctl_cmd
param1	uint32	Configuration parameter 1, determined according to the configuration command

param2	uint32	Configuration parameter 2, determined according to the configuration command
--------	--------	--

Return value

return value	type	illustrate
RET_OK	int32	PWM channel configuration successful
RET_ERR	int32	PWM channel configuration failed

Code examples

```
struct pwm_device *pwm_test = NULL;

pwm_test = (struct pwm_device*)dev_get(HG_PWM_DEVID);

/* Get the PWM handle through dev_get(), use channel 0,
   Configure period value 100us (10KHz frequency) and high level time (90us)
*/

pwm_init(pwm_test, PWM_CHANNEL_0, 100, 90);

/* Modify the period value and high level duration*/

pwm_stop(pwm_test, PWM_CHANNEL_0, 100, 10);

/* Start PWM channel 0 to output PWM waveform*/

pwm_start(pwm_test, PWM_CHANNEL_0);
```

14.4.6. pwm_request_irq()

According to the interrupt type of enum pwm_irq_flag, call this function to apply for the interrupt of the PWM channel.

Function prototype

```
int32 pwm_request_irq(struct pwm_device *pwm,
                    enum pwm_channel channel,
                    enum pwm_irq_flag irq_flag,
                    pwm_irq_hdl          irq_hdl,
                    uint32                data
                    )
```


Function parameters

parameter	type	illustrate
mod	struct pwm_device	Handle to PWM. Usually use dev_get() to get the handle
channel	enum pwm_channel	PWM channel, refer to enumeration pwm_channel
irq_flag	enum pwm_irq_flag	The requested interrupt type, refer to enumeration pwm_irq_flag
irq_hdl	pwm_irq_hdl	Interrupt handle, executed after the interrupt is generated
data	uint32	Interrupt handle parameters

Return value

return value	type	illustrate
RET_OK	int32	PWM channel application interrupt successful
RET_ERR	int32	PWM channel application interrupt failed

Code examples

```
void pwm_irq_hdl(uint32 irq, uint32 irq_data) {  
  
    __NOP();  
  
}  
  
struct pwm_device *pwm_test = NULL;  
  
pwm_test = (struct pwm_device*)dev_get(HG_PWM_DEVID);  
  
/* Get the PWM handle through dev_get(), use channel 0,  
  
    Configure period value 100us (10KHz frequency) and high level time (90us)  
  
*/  
  
pwm_init(pwm_test, PWM_CHANNEL_0, 100, 90);  
  
/* Apply for the TIMER bound to PWM channel 0 to count to the period value interrupt */  
  
pwm_request_irq(pwm_test, PWM_CHANNEL_0, PWM_IRQ_FLAG_COMPARE, \br/>  
pwm_irq_hdl, 0);  
  
/* Start PWM channel 0 to output PWM waveform*/
```

```
pwm_start(pwm_test, PWM_CHANNEL_0);
```

14.4.7. pwm_release_irq()

According to the interrupt type of enum pwm_irq_flag, call this function to release all interrupts of the PWM channel.

Function prototype

```
int32 pwm_release_irq(struct pwm_device *pwm, enum pwm_channel channel)
```

Function parameters

parameter	type	illustrate
mod	struct pwm_device	Handle to PWM. Usually use dev_get() to get the handle
channel	enum pwm_channel	PWM channel, refer to enumeration pwm_channel

Return value

return value	type	illustrate
RET_OK	int32	PWM channel release interrupt successful
RET_ERR	int32	PWM channel release interrupt failed

Code examples

```
struct pwm_device *pwm_test = NULL;

pwm_test = (struct pwm_device*)dev_get(HG_PWM_DEVID);

/* Get the PWM handle through dev_get(), use channel 0,
   Configure period value 100us (10KHz frequency) and high level time (90us)
*/

pwm_init(pwm_test, PWM_CHANNEL_0, 100, 90);

/* Apply for the TIMER bound to PWM channel 0 to count to the period value interrupt */

pwm_request_irq(pwm_test, PWM_CHANNEL_0, PWM_IRQ_FLAG_COMPARE, \
```

```
pwm_irq_hdl, 0);

/* Release all interrupts of PWM channel 0*/

pwm_release_irq(pwm_test, PWM_CHANNEL_0);

/* Start PWM channel 0 to output PWM waveform*/

pwm_start(pwm_test, PWM_CHANNEL_0);
```

15. CAPTURE interface description

This chapter mainly introduces the functions, enumerations, macros, and structures of CAPTURE. They are located in the "" file directory

15.1. Enum

15.1.1. capture_channel

capture_channel describes the CAPTURE capture channel.

enumerator	illustrate
CAPTURE_CHANNEL_0 ~ 3	Capture channel of CAPTURE @Note: Users must bind the channel to the corresponding TIMER in the device.c file.

15.1.2. capture_mode

capture_mode describes the capture mode.

enumerator	illustrate
CAPTURE_MODE_RISE	CAPTURE Capture rising edge
CAPTURE_MODE_FALL	CAPTURE Capture falling edge
CAPTURE_MODE_ALL	CAPTURE Capture double edges

15.1.3. capture_irq_flag

capture_irq_flag describes the interrupt types that CAPTURE can apply for.

enumerator	illustrate
CAPTURE_IRQ_FLAG_CAPTURE	CAPTURE channel capture signal successfully interrupted
CAPTURE_IRQ_FLAG_OVERFLOW	The CAPTURE channel-bound TIMER counts to the period value interrupt, that is, in the TIMER week The signal was not successfully captured within the period.

15.1.4. capture_ioctl_cmd

capture_ioctl_cmd has not yet defined a useful cmd, so it will not be described.

15.2. Define

15.2.1. MACRO

Describe the function of the macro. If the macro needs to pass parameters, describe it in the table below; if there are no parameters to pass, describe it in the table below.

The form above is filled out. (Note: Macro names are in capital letters)

parameter	type	illustrate
Parameter 1		
Parameter 2		

15.3. Structure

15.3.1. capture_device

This structure describes function pointers and operating system related content, and is not recommended for users to modify.

data element type		illustrate
dev	struct dev_obj	Describes operating system related content
heat	int32 function pointer	
deinit	int32 function pointer	
start	int32 function pointer	
stop	int32 function pointer	
ioctl	int32 function pointer	
request_irq	int32 function pointer	
release_irq	int32 function pointer	

15.4. Function

15.4.1. capture_init()

Initialize the CAPTURE channel. This function must be called before using the CAPTURE module channel. CAPTURE pass

After the channel initialization is successful, the default configuration is: TIMER bound to the CAPTURE channel uses the system clock for calculation by default.
number; after the CAPTURE channel captures the signal successfully, the bound TIMER starts counting from 0 again.

Function prototype

```
int32 capture_init( struct capture_device *capture,
                  enum capture_channel channel,
                  enum capture_mode mode
                  )
```

Function parameters

parameter	type	illustrate
capture	Handle to struct capture_device CAPTURE. Usually use dev_get() to get the handle	
channel	enum capture_channel CAPTURE channel, refer to enumeration capture_channel	
mode	enum capture_mode	CAPTURE channel capture mode, refer to enumeration capture_mode

Return value

return value	type	illustrate
RET_OK	int32	CAPTURE channel initialization successful
RET_ERR	int32	CAPTURE channel initialization failed

Code examples

```
struct capture_device *capture_test = NULL;

capture_test = (struct capture_device*)dev_get(HG_CAPTURE_DEVID);

/* Obtain the handle of CAPTURE through dev_get(), use channel 0,

    Configure capture rising edge

*/

capture_init(capture_test, CAPTURE_CHANNEL_0, CAPTURE_MODE_RISE);
```

15.4.2. capture_deinit()

Close the CAPTURE channel. After calling this function, the CAPTURE channel will not work properly and needs to be reused.

The capture_init() function performs initialization.

Function prototype

```
int32 capture_deinit( struct capture_device *capture,

                    enum capture_channel channel

                    )
```

Function parameters

parameter	type	illustrate
capture	Handle to struct capture_device	CAPTURE. Usually use dev_get() to get the handle
channel	enum capture_channel	CAPTURE channel, refer to enumeration capture_channel

Return value

return value	type	illustrate
RET_OK	int32	CAPTURE channel closed successfully
RET_ERR	int32	CAPTURE channel closing failed

Code examples

```
struct capture_device *capture_test = NULL;

capture_test = (struct capture_device*)dev_get(HG_CAPTURE_DEVID);

/* Obtain the handle of CAPTURE through dev_get(), use channel 0,

    Configure capture rising edge

*/

capture_init(capture_test, CAPTURE_CHANNEL_0, CAPTURE_MODE_RISE);

/* Close CAPTURE channel 0 */

capture_deinit(capture_test, CAPTURE_CHANNEL_0);
```

15.4.3. capture_start()

Start the CAPTURE channel for capture.

Note:

- 1. The capture value requires the user to apply for a successful interruption of the CAPTURE channel capture signal and obtain it in the callback value.

Function prototype

```
int32 capture_start(struct capture_device *capture,

                  enum capture_channel channel

                  )
```

Function parameters

parameter	type	illustrate
capture	Handle to struct capture_device	CAPTURE. Usually use dev_get() to get the handle
channel	enum capture_channel	CAPTURE channel, refer to enumeration capture_channel

Return value

return value	type	illustrate
RET_OK	int32	CAPTURE channel started successfully
RET_ERR	int32	CAPTURE channel startup failed

Code examples


```
struct capture_device *capture_test = NULL;

capture_test = (struct capture_device*)dev_get(HG_CAPTURE_DEVID);

/* Obtain the handle of CAPTURE through dev_get(), use channel 0,

    Configure capture rising edge

*/

capture_init(capture_test, CAPTURE_CHANNEL_0, CAPTURE_MODE_RISE);

/* Start CAPTURE channel 0 for capture*/

capture_start(capture_test, CAPTURE_CHANNEL_0);
```

15.4.4. capture_stop()

Stop the CAPTURE channel to capture.

Function prototype

```
int32 capture_stop( struct capture_device *capture,

                    enum capture_channel channel

                    )
```

Function parameters

parameter	type	illustrate
capture	Handle to struct capture_device CAPTURE. Usually use dev_get() to get the handle	
channel	enum capture_channel CAPTURE channel, refer to enumeration capture_channel	

Return value

return value	type	illustrate
RET_OK	int32	CAPTURE channel stopped successfully
RET_ERR	int32	CAPTURE channel stop failed

Code examples

```
struct capture_device *capture_test = NULL;

capture_test = (struct capture_device*)dev_get(HG_CAPTURE_DEVID);

/* Obtain the handle of CAPTURE through dev_get(), use channel 0,

    Configure capture rising edge

*/

capture_init(capture_test, CAPTURE_CHANNEL_0, CAPTURE_MODE_RISE);

/* Start CAPTURE channel 0 for capture*/

capture_start(capture_test, CAPTURE_CHANNEL_0);

/* Stop CAPTURE channel 0 for capturing*/

capture_stop(capture_test, CAPTURE_CHANNEL_0);
```

15.4.5. capture_ioctl()

According to the command in enum capture_ioctl_cmd, call this function to configure the CAPTURE channel.

Note:

- 1. The corresponding ioctl cmd has not been set yet, so calling this function is invalid.

Function prototype

```
int32 capture_ioctl(struct capture_device *capture,

                    enum capture_channel channel,

                    enum capture_ioctl_cmd cmd,

                    uint32 param1,

                    uint32 param2

                    )
```

Function parameters

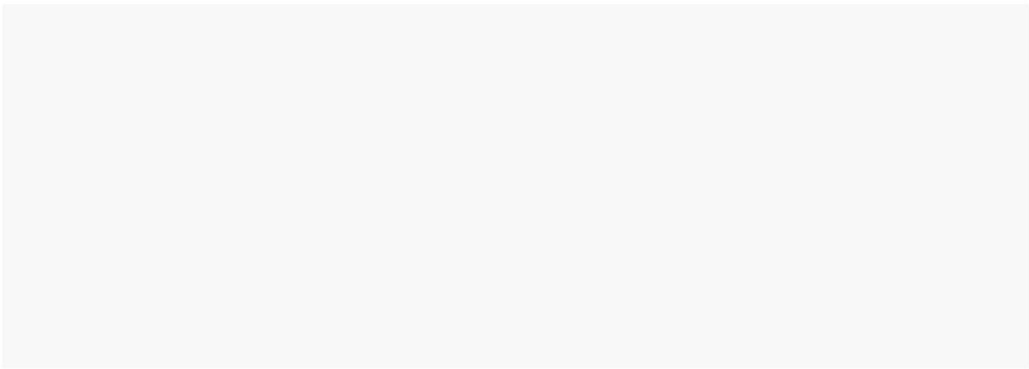
parameter	type	illustrate
capture	struct capture_device	The handle to CAPTURE. Usually use dev_get() to get the handle
channel	enum capture_channel	CAPTURE channel, refer to enumeration capture_channel
cmd	enum capture_ioctl_cmd	CAPTURE channel configuration command, refer to enumeration capture_ioctl_cmd
param1	uint32	Configuration parameter 1, determined according to the configuration command

param2	uint32	Configuration parameter 2, determined according to the configuration command
--------	--------	--

Return value

return value	type	illustrate
RET_OK	int32	CAPTURE channel configuration successful
RET_ERR	int32	CAPTURE channel configuration failed

Code examples



15.4.6. capture_request_irq()

According to the interrupt type of enum capture_irq_flag, call this function to apply for the interrupt of CAPTURE channel.

Function prototype

```
int32 capture_request_irq(struct capture_device *capture,
                        enum capture_channel channel,
                        enum capture_irq_flag irq_flag,
                        capture_irq_hdl          irq_hdl,
                        uint32                    data
                        )
```

Function parameters

parameter	type	illustrate
capture	Handle to struct capture_device CAPTURE. Usually use dev_get() to get the handle	
channel	enum capture_channel CAPTURE channel, refer to enumeration capture_channel	

irq_flag	enum capture_irq_flag	The interrupt type requested, refer to the enumeration capture_irq_flag
irq_hdl	capture_irq_hdl	Interrupt handle, executed after the interrupt is generated
data	uint32	Interrupt handle parameters

Return value

return value	type	illustrate
RET_OK	int32	CAPTURE The channel application was interrupted successfully.
RET_ERR	int32	CAPTURE channel application interrupt failed

Code examples

```
void capture_irq_hdl(uint32 irq, uint32 irq_data) {  
  
    __NOP();  
  
}  
  
struct capture_device *capture_test = NULL;  
  
capture_test = (struct capture_device*)dev_get(HG_CAPTURE_DEVID);  
  
/* Obtain the handle of CAPTURE through dev_get(), use channel 0,  
  
    Configure capture rising edge  
  
*/  
  
capture_init(capture_test, CAPTURE_CHANNEL_0, CAPTURE_MODE_RISE);  
  
/* Apply for CAPTURE channel 0 and interrupt if capture is successful*/  
  
capture_request_irq(capture_test, CAPTURE_CHANNEL_0,\n  
CAPTURE_IRQ_FLAG_CAPTURE, capture_irq_hdl, 0);  
  
/* Start CAPTURE channel 0 for capture*/  
  
capture_start(capture_test, CAPTURE_CHANNEL_0);
```

15.4.7. capture_release_irq()

Based on the interrupt type of enum capture_irq_flag, call this function to release all interrupts of the CAPTURE channel.

Function prototype

```
int32 capture_release_irq(struct capture_device *capture,
                          enum capture_channel channel
                          )
```

Function parameters

parameter	type	illustrate
capture	Handle to struct capture_device	CAPTURE. Usually use dev_get() to get the handle
channel	enum capture_channel	CAPTURE channel, refer to enumeration capture_channel

Return value

return value	type	illustrate
RET_OK	int32	CAPTURE channel release interrupt successful
RET_ERR	int32	CAPTURE channel release interrupt failed

Code examples

```
void capture_irq_hdl(uint32 irq, uint32 irq_data) {
    __NOP();
}

struct capture_device *capture_test = NULL;

capture_test = (struct capture_device*)dev_get(HG_CAPTURE_DEVID);

/* Obtain the handle of CAPTURE through dev_get(), use channel 0,
   Configure capture rising edge
*/

capture_init(capture_test, CAPTURE_CHANNEL_0, CAPTURE_MODE_RISE);

/* Apply for CAPTURE channel 0 and interrupt if capture is successful*/

capture_request_irq(capture_test, CAPTURE_CHANNEL_0, \
CAPTURE_IRQ_FLAG_CAPTURE, capture_irq_hdl, 0);

/* Release all interrupts on CAPTURE channel 0*/

capture_release_irq(capture_test, CAPTURE_CHANNEL_0);
```

```
/* Start CAPTURE channel 0 for capture*/  
capture_start(capture_test, CAPTURE_CHANNEL_0);
```